

Assignment 2: The Socket Exchange — Experiment Report

Course COL334 — Computer Networks

Submitted by Vijeta Khillare 2024CS10573 Harshit Kumar 2024CS10836

Date 10 September 2026

Contents

- 0. Environment, Build, and Methodology
 - 1. Experiment 1 — Listening and Connected Sockets
 - 2. Experiment 2 — Observing TCP Connection States
 - 3. Experiment 3 — TCP as a Byte Stream
 - 4. Experiment 4 — One Client Should Not Stall the Others
 - 5. Experiment 5 — Multiple Clients and I/O Multiplexing (*optional*)
 - 6. Experiment 6 — FIN vs. RST: Orderly and Abrupt Connection Termination
 - 7. Experiment 7 — Backpressure and the Slow Receiver
 - 8. Experiment 8 — Unexpected Client Disconnection
 - 9. Section 6.9 — Bonus: Connection Scalability and I/O Design
- 10. Implementation Decisions
- 11. Appendix A — Tools and What Each One Observes
- 12. Appendix B — Reproduction Instructions

0. Environment, Build, and Methodology

0.1 Platform

Item	Value
Operating system	FreeBSD 14.4-RELEASE
Architecture	ARM64 (aarch64), Apple Silicon host under UTM
VM resources	2 vCPU, 2 GB RAM, no X11
Compiler	<code>c++</code> (clang), C++17, <code>-O2 -Wall -Wextra -Wpedantic</code>
Server concurrency model	single-threaded, non-blocking sockets, <code>kqueue</code> event loop
Client concurrency model	blocking sockets + <code>select()</code> multiplexing stdin and the socket
Transport	TCP over the loopback interface <code>lo0</code> , <code>127.0.0.1:5000</code>

Because the VM is aarch64, kernel stack traces show `do_el0_sync` / `handle_el0_sync` as the syscall entry frames rather than the `amd64_syscall` / `fast_syscall_common` frames seen on x86-64. All

stack evidence in this report should be read with that in mind.

0.2 Build and launch

```
make                                # builds exchange_server,
trader_client, market_data_client

# The three launcher scripts must be executable. Archive formats do not
always
# preserve the executable bit, so if a launcher reports "Permission
denied":
chmod +x server/run-server client/run-trader client/run-market-data

./server/run-server 127.0.0.1 5000    # add -v, or set SX_VERBOSE=1, for
per-recv logging
./client/run-trader 127.0.0.1 5000 alice
./client/run-market-data 127.0.0.1 5000 JNST
```

This was encountered in practice: after unpacking the submission tree, the client launchers failed with a permission error until `chmod +x` was applied. The launchers are shipped with the executable bit set, and the same `chmod` line is documented in `README.md`.

The launcher scripts use `exec`, so the PID reported by the harness as `Started Exchange Server (PID N)` is the `exchange_server` process. This was verified with `procstat -b <pid>` and is what makes every `procstat/ktrace` measurement in this report attributable to the server itself rather than to an intermediate shell.

0.3 Instrumentation added for the experiments

Two diagnostic log statements were added to the server. Both are gated behind a verbosity flag, so the submitted server is silent by default and behaves identically to an uninstrumented build:

Flag	Effect	Used by
<code>SX_VERBOSE=1</code> (env) or <code>-v</code> (argv)	logs <code>[recv] fd=N n=B data=...</code> for every <code>recv()</code>	Experiment 3
same flag	logs <code>[flush] fd=N send() EAGAIN, B bytes still queued</code>	Experiment 7

The environment-variable form exists because `experiment.py` invokes the server as `./server/run-server 127.0.0.1 5000` with no additional arguments, so a command-line flag can never reach it. The environment is inherited through `Popen`, so `SX_VERBOSE=1 python3 experiment.py N` propagates correctly.

One further change was made to `src/netutil.hpp`: the listen backlog was raised from `SOMAXCONN` (the compile-time constant 128) to 4096, because the bonus connection generator opens connections faster than a 128-deep accept queue can absorb. The kernel clamps this value down to `kern.ipc.soacceptqueue`, so it is safe on an untuned machine.

0.4 Kernel baseline

All experiments except Experiment 7 and the Bonus were run against stock FreeBSD 14.4 tunables. Where a sysctl was changed, the change is stated in that experiment's section together with the before value, and it was restored before the next experiment.

```
sysctl net.inet.tcp.msl net.inet.tcp.sendspace net.inet.tcp.recvspace \
net.inet.tcp.sendbuf_auto net.inet.tcp.recvbuf_auto \
net.inet.tcp.keepidle \
net.inet.tcp.always_keepalive net.inet.ip.portrange.first \
net.inet.ip.portrange.last kern.ipc.maxsockbuf kern.ipc.somaxconn \
kern.maxfiles kern.maxfilesperproc
```

The recorded defaults for this VM are shown below; every deviation from these values is stated in the experiment that required it, together with the command used to restore it.

```
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # sysctl net.inet.tcp.msl \
> net.inet.tcp.finwait2_timeout \
> net.inet.tcp.fast_finwait2_recycle \
> net.inet.tcp.sendspace net.inet.tcp.recvspace \
> net.inet.tcp.sendbuf_auto net.inet.tcp.recvbuf_auto \
> net.inet.tcp.sendbuf_max net.inet.tcp.recvbuf_max \
> net.inet.tcp.keepidle net.inet.tcp.keepintvl net.inet.tcp.always_keepalive \
> kern.ipc.maxsockbuf kern.maxfiles kern.maxfilesperproc
net.inet.tcp.msl: 30000
net.inet.tcp.finwait2_timeout: 60000
net.inet.tcp.fast_finwait2_recycle: 0
net.inet.tcp.sendspace: 32768
net.inet.tcp.recvspace: 65536
net.inet.tcp.sendbuf_auto: 1
net.inet.tcp.recvbuf_auto: 1
net.inet.tcp.sendbuf_max: 2097152
net.inet.tcp.recvbuf_max: 2097152
net.inet.tcp.keepidle: 720000
net.inet.tcp.keepintvl: 75000
net.inet.tcp.always_keepalive: 1
kern.ipc.maxsockbuf: 2097152
kern.maxfiles: 64583
kern.maxfilesperproc: 58122
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # sysctl -a | grep -i timewait
net.inet.tcp.nolocaltimewait: 1
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #
```

0.5 A recurring artifact: the harness readiness probe

`experiment.py`'s `wait_for_server()` opens a probe connection to confirm the server is listening and then closes it abortively with `SO_LINGER{onoff=1, linger=0}`. Consequently **every experiment begins with a spurious connection followed by an RST**, and the server log always opens with a line of the form:

```
[server] accepted fd=5 owner=1 from 127.0.0.1:<port>
[server] fd=5: connection reset by peer (RST, ECONNRESET)
[server] closing fd=5 owner=1
```

This is not part of any experiment and has been excluded from all analysis below. It does, however, independently confirm that the server's `ECONNRESET` handling path works, which is the finding of Experiment 6 Part B.

0.6 Standard terminal layout

Every experiment used the same arrangement, with T2–T4 started **before** the harness so that no transient state was missed:

Terminal	Role
T1	<code>experiment.py</code> harness and inherited server stderr
T2	kernel TCP state poller (<code>poll-tcp.sh</code> / <code>poll-tcp.x.sh</code> , 0.1–0.5 s, <code>tee</code> 'd to a log)
T3	packet capture (<code>tcpdump -i lo0</code> , redirected or <code>tee</code> 'd to a text log)
T4	syscall tracing (<code>ktrace</code> before the run, <code>kdump</code> after)
T5	active inspection during the run (<code>netstat</code> , <code>sockstat</code> , <code>procstat -f</code> , <code>procstat -k</code>)

T4 and T5 are separate because `ktrace` must be attached *before* the server spawns and detached *after* it exits, which occupies that terminal for the whole run; anything that must be sampled while the harness is mid-phase therefore needs its own tab.

0.6.1 Attaching to the server the instant it spawns

The harness starts the server itself, so tracing cannot simply be started beforehand against a known PID. Every experiment that uses `ktrace` opened T4 with the same spin-wait, which attaches within 50 ms of the process appearing:

```
while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done
ktrace -i -f /tmp/expN.kt -p $PID -t c
```

The PID is captured **once** into a variable here. Re-running `pgrep` inline at sampling time is unreliable: if the server has already exited, the substitution is empty and a command such as `procstat -k $(pgrep -n exchange_server)` degenerates into a bare `procstat -k` and prints a usage message instead of a stack.

0.6.2 Hygiene between runs

```
kill exchange_server
ktrace -C                # release any trace lock left by a previous
run                      #
pgrep -f exchange_server # must print nothing before the next
experiment
```


1. Experiment 1 — Listening and Connected Sockets

Question. After the client has connected, identify the TCP sockets associated with the Exchange Server. How does the listening socket differ from the socket representing the connection to the client?

Approach

The harness establishes one connection and holds it idle indefinitely, so this is a static-state experiment. The strategy was to observe the same two sockets through three independent kernel views — the protocol control block table, the process-to-socket mapping, and the per-process file-descriptor table — and show that all three agree on the same distinction.

Commands and tools

```
# T2 – kernel state poller
~/tools/poll-tcp.sh 1

# T3 – packet capture
tcpdump -i lo0 -n -S -tttt -vv 'tcp port 5000' | tee /tmp/exp1.txt

# T1 – harness (client connects and stays idle until Ctrl-C)
python3 experiment.py 1 2>&1 | tee /tmp/exp1.log

# T4 – inspection, while the connection is established
PID=$(pgrep -n exchange_server)
netstat -an -p tcp | egrep 'Proto|\.5000'      # protocol view
sockstat -4 -p 5000                          # process ↔ socket view
procstat -f $PID | egrep 'PID|TCP|kqueue'     # file-descriptor view
```

Supplementary checks used to confirm the passive/active distinction:

```
procstat -b $PID          # confirms exec() replaced the launcher shell
sockstat -4 -l -p 5000    # listening sockets only
sockstat -4 -c -p 5000    # connected sockets only
netstat -aLn -p tcp       # listen queue depth for 127.0.0.1.5000
```

Screenshots

```

root@exchange:/home/cooluser1 # PID=$(pgrep -n exchange_server); echo "Server PID is: $PID"
Server PID is: 2432
root@exchange:/home/cooluser1 # echo "--- 1. Exec Verification ---"
--- 1. Exec Verification ---
root@exchange:/home/cooluser1 # procstat -b $PID
  PID COMM          OSREL  PATH
  2432 exchange_server  1404000 /home/cooluser1/A2_2024CS10573_2024CS10836/exchange_server
root@exchange:/home/cooluser1 #
root@exchange:/home/cooluser1 # echo "--- 2. Protocol View (netstat) ---"
--- 2. Protocol View (netstat) ---
root@exchange:/home/cooluser1 # netstat -an -p tcp | egrep 'Proto|\.5000'
Proto      Recv-Q Send-Q Local Address           Foreign Address         (state)
tcp4        0      0 127.0.0.1.5000          127.0.0.1.26623        ESTABLISHED
tcp4        0      0 127.0.0.1.26623         127.0.0.1.5000         ESTABLISHED
tcp4        0      0 127.0.0.1.5000          *.*                     LISTEN
root@exchange:/home/cooluser1 #
root@exchange:/home/cooluser1 # echo "--- 3. Process View (sockstat) ---"
--- 3. Process View (sockstat) ---
root@exchange:/home/cooluser1 # sockstat -4 -p 5000
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
root      exchange_s 2432 3   tcp4   127.0.0.1:5000  *.*
root      exchange_s 2432 5   tcp4   127.0.0.1:5000  127.0.0.1:26623
root      python3.12 2430 3   tcp4   127.0.0.1:26623  127.0.0.1:5000
root@exchange:/home/cooluser1 #
root@exchange:/home/cooluser1 # echo "--- 4. File Descriptor View (procstat -f) ---"
--- 4. File Descriptor View (procstat -f) ---
root@exchange:/home/cooluser1 # procstat -f $PID | egrep 'PID|TCP|kqueue'
  PID COMM          FD T V FLAGS  REF  OFFSET PRO NAME
  2432 exchange_server  3 s - rw---n--  2      0 TCP 0 0 127.0.0.1:5000 *:0
  2432 exchange_server  5 s - rw---n--  3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:26623
root@exchange:/home/cooluser1 #
root@exchange:/home/cooluser1 # echo "--- 5. Listen Queue ---"
--- 5. Listen Queue ---
root@exchange:/home/cooluser1 # netstat -aLn -p tcp | egrep 'Proto|\.5000'
Proto Listen                                Local Address
tcp4      0/0/128                                127.0.0.1.5000
root@exchange:/home/cooluser1 #

```

Observation

Property	Listening socket (fd 3)	Connected socket (fd 5)
Local address	127.0.0.1:5000	127.0.0.1:5000
Foreign address	*.* (wildcard)	127.0.0.1:26623 (specific)
State	LISTEN	ESTABLISHED
Identified by	half-association (local endpoint only)	full 4-tuple
Carries application data	no — holds the SYN and accept queues	yes
Recv-Q / Send-Q	not byte counters; backlog shown by netstat -aLn as 0/0/128	byte counters, 0 / 0 while idle

- Protocol view (netstat)** — three rows exist for port 5000: the listener, the server-side connected socket, and the client-side connected socket. The client-side row is the mirror image of the server-side row, which is a direct consequence of both endpoints living in the same VM.
- Process view (sockstat)** — **-l** and **-c** cleanly separate the two, attributing both to the same PID.
- File-descriptor view (procstat -f)** — the server holds fd 3 (listener, no foreign endpoint), fd 4 (the **kqueue** descriptor itself), and fd 5 (the accepted connection, full 4-tuple). The presence of a

single `kqueue` descriptor alongside every connection descriptor is the file-descriptor-level signature of the I/O design described in §10.

Note that the accepted connection received fd 5 rather than fd 4, because fd 4 was already taken by the `kqueue` instance created at startup, and the descriptor used by the harness's readiness probe had already been closed and its number recycled.

Answer

The listening socket is a passive endpoint: it is bound to a local address and port with a wildcard peer, transfers no application data, and exists only so that `accept()` can return a fresh descriptor for each completed handshake. The connected socket is identified by the complete 4-tuple `127.0.0.1:5000 ↔ 127.0.0.1:26623` and is the only one of the two over which bytes flow.

Both sockets share the same local address and port, which demonstrates that a local endpoint alone does not identify a connection. The kernel demultiplexes an arriving segment by searching its protocol-control-block table for an exact 4-tuple match first, falling back to the wildcard listener only for a segment matching no established connection — that is, for a SYN. This is precisely why a single bound port can back arbitrarily many simultaneous connections, each with its own descriptor, its own socket buffers, and its own `Conn` object in the server.

2. Experiment 2 — Observing TCP Connection States

Question. How does the TCP state of the client-server connection change during its lifetime, and what events cause the observed state changes?

Approach

Teardown on loopback completes in microseconds, so no polling interval can reliably capture every intermediate state. A two-layer strategy was therefore used: `tcpdump` provides the authoritative packet-level timeline, which cannot miss an event, and a 0.1 s `netstat` poller captures whichever kernel states persist long enough to observe. The experiment was run twice — once with default tunables, and once with `net.inet.tcp.nolocaltimewait=0` — to determine whether the loopback `TIME_WAIT` suppression optimisation was responsible for the state being absent from the first run.

Commands and tools

```
# T4 – setup, second run only (record the default before changing it)
sysctl net.inet.tcp.nolocaltimewait
sysctl net.inet.tcp.nolocaltimewait=0
sysctl net.inet.tcp.msl                # 30000 ms → TIME_WAIT ≈ 2×MSL
≈ 60 s

# T2 – high-frequency state poller (0.1 s, to catch transient states)
~/tools/poll-tcp.sh 0.1 /tmp/exp2.states

# T3 – packet capture (-l line-buffers so the redirect is not lost on Ctrl-C)
```

```
tcpdump -l -i lo0 -n -S -vv -tttt 'tcp port 5000' > /tmp/exp2_packets.txt

# T1 – harness
SX_VERBOSE=1 python3 -u experiment.py 2 2>&1 | tee /tmp/exp2.log

# T4 – post-run analysis
netstat -an -p tcp | grep TIME_WAIT
egrep -n 'TIME_WAIT|ESTABLISHED' /tmp/exp2.states | tail -20
cat /tmp/exp2_packets.txt
```

Screenshots


```

root@exchange:/home/cooluser1 # PID=$(pgrep -n exchange_server)
root@exchange:/home/cooluser1 # netstat -an -p tcp | egrep 'Proto|\.5000'
Proto      Recv-Q Send-Q Local Address           Foreign Address         (state)
tcp4        0      0 127.0.0.1.5000          127.0.0.1.41506        ESTABLISHED
tcp4        0      0 127.0.0.1.41506         127.0.0.1.5000         ESTABLISHED
tcp4        0      0 127.0.0.1.5000          *.*                     LISTEN
root@exchange:/home/cooluser1 # procstat -f $PID | grep TCP
12954 exchange_server    3 s - rw---n--    2      0 TCP 0 0 127.0.0.1:5000 *:0
12954 exchange_server    5 s - rw---n--    3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:41506
root@exchange:/home/cooluser1 # sockstat -4 -p 5000
USER      COMMAND  PID  FD  PROTO  LOCAL ADDRESS  FOREIGN ADDRESS
root      exchange_s 12954 3   tcp4   127.0.0.1:5000  *.*
root      exchange_s 12954 5   tcp4   127.0.0.1:5000  127.0.0.1:41506
root      python3.12 12948 3   tcp4   127.0.0.1:41506  127.0.0.1:5000
root@exchange:/home/cooluser1 # netstat -an -p tcp | egrep 'Proto|\.5000'
Proto      Recv-Q Send-Q Local Address           Foreign Address         (state)
tcp4        0      0 127.0.0.1.5000          *.*                     LISTEN
root@exchange:/home/cooluser1 # procstat -f $PID | grep TCP
12954 exchange_server    3 s - rw---n--    2      0 TCP 0 0 127.0.0.1:5000 *:0
root@exchange:/home/cooluser1 # netstat -an -p tcp | egrep 'Proto|\.5000'
Proto      Recv-Q Send-Q Local Address           Foreign Address         (state)
tcp4        0      0 127.0.0.1.5000          *.*                     LISTEN
root@exchange:/home/cooluser1 # procstat -f $PID | grep TCP
12954 exchange_server    3 s - rw---n--    2      0 TCP 0 0 127.0.0.1:5000 *:0
root@exchange:/home/cooluser1 # netstat -an -p tcp | egrep 'Proto|\.5000'
Proto      Recv-Q Send-Q Local Address           Foreign Address         (state)
tcp4        0      0 127.0.0.1.5000          *.*                     LISTEN
root@exchange:/home/cooluser1 # procstat -f $PID | grep TCP
12954 exchange_server    3 s - rw---n--    2      0 TCP 0 0 127.0.0.1:5000 *:0
root@exchange:/home/cooluser1 # netstat -an -p tcp | grep TIME_WAIT
root@exchange:/home/cooluser1 # sysctl net.inet.tcp.msl
net.inet.tcp.msl: 30000
root@exchange:/home/cooluser1 # netstat -an -p tcp | grep TIME_WAIT
root@exchange:/home/cooluser1 # sysctl net.inet.tcp.msl
net.inet.tcp.msl: 30000
root@exchange:/home/cooluser1 # sysctl net.inet.tcp.nolocaltimewait
net.inet.tcp.nolocaltimewait: 1
root@exchange:/home/cooluser1 #

```

```

root@exchange:/home/cooluser1 # sysctl net.inet.tcp.nolocaltimewait=0
net.inet.tcp.nolocaltimewait: 1 -> 0
root@exchange:/home/cooluser1 # egrep -n 'TIME_WAIT|ESTABLISHED' /tmp/exp2_run2.states | tail -20
1743:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1747:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1751:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1755:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1759:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1763:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1767:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1771:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1775:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1779:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1783:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1787:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1791:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1795:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1799:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1803:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1807:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1811:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1815:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
1819:tcp4      0      0 127.0.0.1.50394         127.0.0.1.5000        TIME_WAIT
root@exchange:/home/cooluser1 # tail -n 15 /tmp/exp2_run2.txt

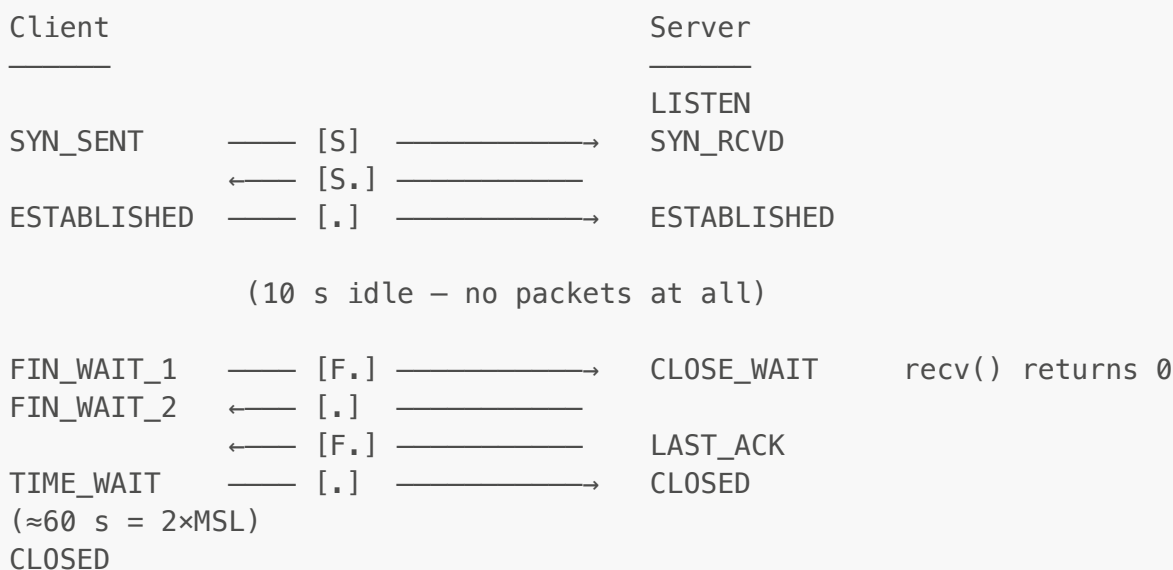
```

```

[root@exchange:/home/cooluser1 # cat /tmp/exp2_packets.txt
2026-09-09 05:36:16.374364 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3c3b))
 127.0.0.1.16396 > 127.0.0.1.5000: Flags [S], cksum 0xfe30 (incorrect -> 0xd79a), seq 2522876695, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2402144264 ecr 0], length 0
2026-09-09 05:36:16.374389 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3c3b))
 127.0.0.1.5000 > 127.0.0.1.16396: Flags [R.], cksum 0xfe1c (incorrect -> 0xd8c3), seq 0, ack 2422076696, win 0, length 0
2026-09-09 05:36:16.478557 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3c3b))
 127.0.0.1.18888 > 127.0.0.1.5000: Flags [S], cksum 0xfe30 (incorrect -> 0x282d), seq 2252824699, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3860512127 ecr 0], length 0
2026-09-09 05:36:16.478689 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3c3b))
 127.0.0.1.5000 > 127.0.0.1.18888: Flags [S.], cksum 0xfe30 (incorrect -> 0x63bd), seq 3734320936, ack 2252824700, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4102797844 ecr 3860512127], length 0
2026-09-09 05:36:16.478708 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
 127.0.0.1.18888 > 127.0.0.1.5000: Flags [R.], cksum 0xfe28 (incorrect -> 0xc7af), seq 2252824700, ack 3734320937, win 1277, options [nop,nop,TS val 3860512127 ecr 4102797844], length 0
2026-09-09 05:36:16.478808 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
 127.0.0.1.18888 > 127.0.0.1.5000: Flags [R.], cksum 0xfe28 (incorrect -> 0xcce8), seq 2252824700, ack 3734320937, win 0, options [nop,nop,TS val 3860512127 ecr 4102797844], length 0
2026-09-09 05:36:16.478942 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3c3b))
 127.0.0.1.45102 > 127.0.0.1.5000: Flags [S], cksum 0xfe30 (incorrect -> 0x1626), seq 1570087077, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1230958180 ecr 0], length 0
2026-09-09 05:36:16.478949 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3c3b))
 127.0.0.1.5000 > 127.0.0.1.45102: Flags [S.], cksum 0xfe30 (incorrect -> 0xe847), seq 580617684, ack 1570087078, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 1126646326 ecr 1230958180], length 0
2026-09-09 05:36:16.478950 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
 127.0.0.1.45102 > 127.0.0.1.5000: Flags [R.], cksum 0xfe28 (incorrect -> 0x4c3a), seq 1570087078, ack 580617685, win 1277, options [nop,nop,TS val 1230958180 ecr 1126646326], length 0
2026-09-09 05:36:26.504622 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
 127.0.0.1.45102 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0x250f), seq 1570087078, ack 580617685, win 1277, options [nop,nop,TS val 1230968206 ecr 1126646326], length 0
2026-09-09 05:36:26.504650 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
 127.0.0.1.5000 > 127.0.0.1.45102: Flags [F.], cksum 0xfe28 (incorrect -> 0xfde4), seq 580617685, ack 1570087079, win 1277, options [nop,nop,TS val 1126656352 ecr 1230968206], length 0
2026-09-09 05:36:26.504813 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
 127.0.0.1.5000 > 127.0.0.1.45102: Flags [F.], cksum 0xfe28 (incorrect -> 0xfde3), seq 580617685, ack 1570087079, win 1277, options [nop,nop,TS val 1126656352 ecr 1230968206], length 0
2026-09-09 05:36:26.504825 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
 127.0.0.1.45102 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0xfde3), seq 1570087079, ack 580617686, win 1277, options [nop,nop,TS val 1230968206 ecr 1126656352], length 0

```

Observation



First run (stock tunables). Both endpoints showed ESTABLISHED, then both disappeared from `netstat` immediately on teardown; no TIME_WAIT was captured at any polling interval.

Second run (`net.inet.tcp.nolocaltimewait=0`). The OID `net.inet.tcp.nolocaltimewait` is present on this FreeBSD 14.4 system and is **enabled by default**, which is what suppressed the state in the first run: it is an optimisation that skips TIME_WAIT for connections whose peer is local, on the grounds that a loopback connection cannot receive a delayed duplicate from the network. With it disabled, repeated polling captured TIME_WAIT persisting for approximately 60 seconds on the client-side ephemeral port 50394.

This pair of runs is the point of the experiment: the missing state in the first run was a deliberate kernel optimisation, not a measurement failure, and disabling one sysctl makes the textbook state machine visible without changing anything in the application.

Packet trace. The connection on ephemeral port 45102 shows the complete sequence: `[S] → [S.] → [.]`, ten seconds of total silence, then `[F.] → [.] → [F.] → [.]`.

Two states could not be photographed, and this is expected. `CLOSE_WAIT` and `LAST_ACK` exist on the server for only microseconds, because the event loop reacts to `recv() == 0` within the same `kevent()` wakeup that reported it: the descriptor is closed before any poller can sample it. That the states are nevertheless traversed is established from the syscall order captured with `kt race`, which

shows `RET recvfrom 0` followed immediately by `CALL shutdown(fd, SHUT_WR)` and `CALL close(fd)` on the same descriptor. A server that deliberately lingered after EOF would hold `CLOSE_WAIT` observably; ours does not, by design.

Answer

The connection's state changes are driven entirely by observed segments and by one timer:

1. **LISTEN** — the server socket before any client arrives.
2. **Three-way handshake** (`[S] → [S.] → [.]`) moves both endpoints to **ESTABLISHED**.
3. **ESTABLISHED persists through ten seconds of complete silence**. TCP neither polls nor probes an idle connection; the absence of packets in the trace during this window is itself the evidence.
4. **Client close()** sends FIN: client **ESTABLISHED** → **FIN_WAIT_1**, server **ESTABLISHED** → **CLOSE_WAIT**, and the server application observes EOF as `recv() == 0`.
5. **Server close()** sends its own FIN: server **CLOSE_WAIT** → **LAST_ACK** → **CLOSED**, client **FIN_WAIT_1** → **FIN_WAIT_2** → **TIME_WAIT** on receiving it.
6. **The 2×MSL timer** (`net.inet.tcp.msl = 30000 ms`, so ≈ 60 s) finally destroys the client-side PCB. **TIME_WAIT** is owned by the kernel, not the process, which is why it remains visible in `netstat` after the harness has exited. Its purpose is to ensure a delayed duplicate segment from this connection cannot be misapplied to a later connection reusing the same 4-tuple.

3. Experiment 3 — TCP as a Byte Stream

Question. Does the Exchange Server receive the application-level message as one complete unit, or can the message be received in multiple pieces? Explain what this demonstrates about the relationship between TCP and application-level message boundaries.

Approach

The harness transmits the single 24-byte application message `LOGIN experiment_trader\n` using four separate `send()` calls — `"LOGIN "` (6 B), `"experiment"` (10 B), `"_trader"` (7 B), `"\n"` (1 B) — spaced 0.2 s apart. The 0.2 s spacing guarantees the segments cannot coalesce, making the result deterministic. Evidence was gathered on both sides of the kernel boundary simultaneously: four discrete segments on the wire, and four discrete `recv()` returns in the application.

Commands and tools

```
# T4 – attach the syscall trace the instant the server spawns
while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done
ktrace -i -f /tmp/exp3.kt -p $PID -t c

# T2 – state poller (to show Recv-Q stays at 0)
~/tools/poll-tcp.sh 0.1

# T3 – packet capture
tcpdump -l -i lo0 -n -S -vv -tttt 'tcp port 5000' | tee /tmp/exp3.txt
```



```
# T1 – harness, with per-recv logging enabled
SX_VERBOSE=1 python3 -u experiment.py 3 2>&1 | tee /tmp/exp3.log

# T4 – post-run extraction
ktrace -C
kdump -f /tmp/exp3.kt -T | egrep 'kevent|recvfrom|sendto|write' | head -60
```

Screenshots

```

harshitkumar — T1 — ssh -p 2222 cooluser1@127.0.0.1 — 217x49
[root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # SX_VERBOSE=1 python3 -u experiment.py 3 2>&1 | tee /tmp/exp3.log
=== Experiment 3: TCP as a Byte Stream ===

Started Exchange Server (PID 26795).
[server] listening on 127.0.0.1:5000 (fd=3, kq=4)
[server] accepted fd=5 owner=1 from 127.0.0.1:29066
Exchange Server is listening.

[server] fd=5: connection reset by peer (RST, ECONNRESET)
[server] closing fd=5 owner=1
Experiment client: connected from 127.0.0.1:11250

Sending one application-level message using several separate TCP writes...[server] accepted fd=5 owner=2 from 127.0.0.1:11250

Sent 6 bytes.
[recv] fd=5 n=6 data=LOGIN
Sent 10 bytes.[recv] fd=5 n=10 data=experiment

Sent 7 bytes.
[recv] fd=5 n=7 data=_trader
Sent 1 bytes.
[recv] fd=5 n=1 data=

Investigate what the Exchange Server received.
Press Ctrl-C when you are finished.
^C
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #

```

```
=== 05:49:42 ===
```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	.*	LISTEN

```
=== 05:49:42 ===
```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.11250	ESTABLISHED
tcp4	0	0	127.0.0.1.11250	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	.*	LISTEN

```
=== 05:49:42 ===
```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.11250	ESTABLISHED
tcp4	0	10	127.0.0.1.11250	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	.*	LISTEN

```
=== 05:49:42 ===
```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.11250	ESTABLISHED
tcp4	0	0	127.0.0.1.11250	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	.*	LISTEN

```
=== 05:49:42 ===
```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.11250	ESTABLISHED
tcp4	0	7	127.0.0.1.11250	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	.*	LISTEN

```
=== 05:49:42 ===
```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.11250	ESTABLISHED
tcp4	0	0	127.0.0.1.11250	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	.*	LISTEN

```
=== 05:49:42 ===
```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	3	127.0.0.1.5000	127.0.0.1.11250	ESTABLISHED
tcp4	3	0	127.0.0.1.11250	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	.*	LISTEN

```
=== 05:49:42 ===
```

```

127.0.0.1.11250 > 127.0.0.1.5000: Flags [P.], cksum 0xfe2e (incorrect -> 0x17c5), seq 2801994884:2801994890, ack 1253854141, win 1277, options [nop,nop,TS val 125014804 ecr 238320903], length 6
2026-09-09 05:49:42.066999 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
127.0.0.1.5000 > 127.0.0.1.11250: Flags [.] , cksum 0xfe28 (incorrect -> 0xf95e), seq 1253854141, ack 2801994890, win 1277, options [nop,nop,TS val 238320942 ecr 125014804], length 0
2026-09-09 05:49:42.230888 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 62, bad cksum 0 (->3cb8))
127.0.0.1.11250 > 127.0.0.1.5000: Flags [P.], cksum 0xfe32 (incorrect -> 0xd45b), seq 2801994890:2801994900, ack 1253854141, win 1277, options [nop,nop,TS val 125015012 ecr 238320942], length 10
2026-09-09 05:49:42.278232 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
127.0.0.1.5000 > 127.0.0.1.11250: Flags [.] , cksum 0xfe28 (incorrect -> 0xf7bc), seq 1253854141, ack 2801994900, win 1277, options [nop,nop,TS val 238321142 ecr 125015012], length 0
2026-09-09 05:49:42.458062 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 59, bad cksum 0 (->3cbb))
127.0.0.1.11250 > 127.0.0.1.5000: Flags [P.], cksum 0xfe2f (incorrect -> 0x4e9f), seq 2801994900:2801994907, ack 1253854141, win 1277, options [nop,nop,TS val 125015223 ecr 238321142], length 7
2026-09-09 05:49:42.497074 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
127.0.0.1.5000 > 127.0.0.1.11250: Flags [.] , cksum 0xfe28 (incorrect -> 0xf602), seq 1253854141, ack 2801994907, win 1277, options [nop,nop,TS val 238321366 ecr 125015223], length 0
2026-09-09 05:49:42.670926 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 53, bad cksum 0 (->3cc1))
127.0.0.1.11250 > 127.0.0.1.5000: Flags [P.], cksum 0xfe29 (incorrect -> 0xeb18), seq 2801994907:2801994908, ack 1253854141, win 1277, options [nop,nop,TS val 125015448 ecr 238321366], length 1
2026-09-09 05:49:42.671223 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 55, bad cksum 0 (->3cbf))

root@exchange:/home/cooluser1 # while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done; echo "Tracing PID $PID..."; ktrace -i -f /tmp/exp3.kt -p $PID -t c
Tracing PID 26795...
root@exchange:/home/cooluser1 # ktrace -C
root@exchange:/home/cooluser1 # kdump -f /tmp/exp3.kt -T | egrep 'kevent|recvfrom|sendto|write' | head -60
26795 exchange_server 1788913182.027276860 RET kevent 1
26795 exchange_server 1788913182.027380318 CALL kevent(0x4,0x80eeb3e8,0x1,0,0,0)
26795 exchange_server 1788913182.027387735 RET kevent 0
26795 exchange_server 1788913182.027388735 CALL kevent(0x4,0x80eeb3e8,0x1,0,0,0)
26795 exchange_server 1788913182.027390776 RET kevent 0
26795 exchange_server 1788913182.027405360 CALL write(0x2,0x80eeac10,0x34)
26795 exchange_server 1788913182.027409985 RET write 52/0x34
26795 exchange_server 1788913182.027438276 CALL kevent(0x4,0,0,0x80eeb490,0x40,0)
26795 exchange_server 1788913182.027441193 RET kevent 1
26795 exchange_server 1788913182.027468151 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.027473110 RET recvfrom -1 errno 54 Connection reset by peer
26795 exchange_server 1788913182.027475943 CALL write(0x2,0x80ee9c60,0x3a)
26795 exchange_server 1788913182.027477524 RET write 58/0x3a
26795 exchange_server 1788913182.027490235 CALL write(0x2,0x80ee9bf0,0x1e)
26795 exchange_server 1788913182.027491276 RET write 30/0x1e
26795 exchange_server 1788913182.027515485 CALL kevent(0x4,0,0,0x80eeb490,0x40,0)
26795 exchange_server 1788913182.027640776 RET kevent 1
26795 exchange_server 1788913182.027658151 CALL kevent(0x4,0x80eeb3e8,0x1,0,0,0)
26795 exchange_server 1788913182.027661735 RET kevent 0
26795 exchange_server 1788913182.027662693 CALL kevent(0x4,0x80eeb3e8,0x1,0,0,0)
26795 exchange_server 1788913182.027663776 RET kevent 0
26795 exchange_server 1788913182.027668568 CALL write(0x2,0x80eeac10,0x34)
26795 exchange_server 1788913182.027670235 RET write 52/0x34
26795 exchange_server 1788913182.027673818 CALL kevent(0x4,0,0,0x80eeb490,0x40,0)
26795 exchange_server 1788913182.028009776 RET kevent 1
26795 exchange_server 1788913182.028105151 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.028110735 RET recvfrom 6
26795 exchange_server 1788913182.028118526 CALL write(0x2,0x80ee9c60,0x1c)
26795 exchange_server 1788913182.028121068 RET write 28/0x1c
26795 exchange_server 1788913182.028140651 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.028141943 RET recvfrom -1 errno 35 Resource temporarily unavailable
26795 exchange_server 1788913182.028142860 CALL kevent(0x4,0,0,0x80eeb490,0x40,0)
26795 exchange_server 1788913182.239147526 RET kevent 1
26795 exchange_server 1788913182.239168151 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.239177110 RET recvfrom 10/0xa
26795 exchange_server 1788913182.239189776 CALL write(0x2,0x80ee9c60,0x21)
26795 exchange_server 1788913182.239193526 RET write 33/0x21
26795 exchange_server 1788913182.239197818 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.239199485 RET recvfrom -1 errno 35 Resource temporarily unavailable
26795 exchange_server 1788913182.239201235 CALL kevent(0x4,0,0,0x80eeb490,0x40,0)
26795 exchange_server 1788913182.458208110 RET kevent 1
26795 exchange_server 1788913182.458225901 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.458250526 RET recvfrom 7
26795 exchange_server 1788913182.458269943 CALL write(0x2,0x80ee9c60,0x1d)
26795 exchange_server 1788913182.458273735 RET write 29/0x1d
26795 exchange_server 1788913182.458342193 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.458346485 RET recvfrom -1 errno 35 Resource temporarily unavailable
26795 exchange_server 1788913182.458348776 CALL kevent(0x4,0,0,0x80eeb490,0x40,0)
26795 exchange_server 1788913182.671066401 RET kevent 1
26795 exchange_server 1788913182.671083485 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.671110568 RET recvfrom 1
26795 exchange_server 1788913182.671128985 CALL write(0x2,0x80ee9c60,0x17)
26795 exchange_server 1788913182.671132360 RET write 23/0x17
26795 exchange_server 1788913182.671215068 CALL sendto(0x5,0x2a15c2809061,0x3,0,0,0)
26795 exchange_server 1788913182.671257735 RET sendto 3
26795 exchange_server 1788913182.671262235 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
26795 exchange_server 1788913182.671265193 RET recvfrom -1 errno 35 Resource temporarily unavailable
26795 exchange_server 1788913182.671267360 CALL kevent(0x4,0,0,0x80eeb490,0x40,0)
26795 exchange_server 1788913192.177593735 RET kevent 1
26795 exchange_server 1788913192.177612026 CALL recvfrom(0x5,0x80eeac408,0x1000,0,0,0)
root@exchange:/home/cooluser1 #

```

Observation

Wire. Four data segments, each with the PSH flag set, 0.2 s apart, with lengths 6, 10, 7 and 1 and strictly increasing sequence numbers, each individually acknowledged by the server. The server's 3-byte **OK\n** reply appears only after the fourth.

Application (server log).

```

[recv] fd=5 n=6  data=LOGIN
[recv] fd=5 n=10 data=experiment
[recv] fd=5 n=7  data=_trader
[recv] fd=5 n=1  data=

```

Syscalls (kdump). The one-to-one correspondence with the segments, and the **EAGAIN** that terminates each read burst:


```

recvfrom(0x5,...)  RET 6                                ← "LOGIN "
recvfrom(0x5,...)  RET -1 errno 35 Resource temporarily unavailable
... kevent sleeps ~0.2 s ...
recvfrom(0x5,...)  RET 10                               ←
"experiment"
recvfrom(0x5,...)  RET -1 errno 35
... kevent sleeps ~0.2 s ...
recvfrom(0x5,...)  RET 7                                ← "_trader"
recvfrom(0x5,...)  RET -1 errno 35
... kevent sleeps ~0.2 s ...
recvfrom(0x5,...)  RET 1                                ← "\n"

```

Kernel queue. `Recv-Q` reads 0 in every poller snapshot. This is worth emphasising: the partial bytes are **not** waiting in the kernel socket buffer, they have already been copied into the server's userspace `LineBuffer`, where they sit as an incomplete application message. Buffering for framing purposes happens in the application, not in the kernel.

Answer

The message is **not** received as one unit. Four writes produced four TCP segments and four `recv()` returns of 6, 10, 7 and 1 bytes, and the server produced its `OK` response only after the fourth `recv()` delivered the `\n`.

TCP guarantees an ordered, reliable **byte stream** and nothing more. It preserves neither the sender's write boundaries nor any notion of a message, and is free to split or coalesce data at any point in the stream. Application message boundaries therefore exist only in the application, which is why the server accumulates bytes in a per-connection `LineBuffer` that persists across `recv()` calls and extracts messages by scanning for the `\n` delimiter in a loop. The same loop handles the converse case — several complete messages arriving in a single `recv()` — which is why extraction is written as `while (next_line(...) == 0k)` rather than as a single call.

4. Experiment 4 — One Client Should Not Stall the Others

Question. When Client 1 remains connected but sends no data, can the Exchange Server still accept and service Client 2? Use your observations to identify the server operation that determines the answer.

Approach

Client 1 connects and sends `LOGIN blocked_client` — 20 bytes, deliberately **without** a trailing newline — then goes silent permanently. Two seconds later Client 2 connects and sends a complete `LOGIN`, and the harness times the round trip. The question is *where the server's single thread is*, so the server's kernel stack was sampled repeatedly across the entire run, corroborated by a syscall trace showing which descriptors the event loop actually acted on.

Commands and tools

```
# T4 – attach the syscall trace as the server spawns
while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done
ktrace -i -f /tmp/exp4.kt -p $PID -t c

# T2 – state poller
~/tools/poll-tcp.sh 0.2

# T3 – packet capture
tcpdump -l -i lo0 -n -S -tttt -vv 'tcp port 5000' > /tmp/exp4_packets.txt

# T1 – harness
SX_VERBOSE=1 python3 -u experiment.py 4 2>&1 | tee /tmp/exp4.log

# T5 – kernel stack sampler, started the moment T1 begins and left running
PID=$(pgrep -n exchange_server)
while ;; do date '+%H:%M:%S'; procstat -k $PID; sleep 0.5; done >
/tmp/exp4.kstack

# T4 – post-run analysis
ktrace -C
cat /tmp/exp4_packets.txt
kdump -f /tmp/exp4.kt -T | egrep 'kevent|recvfrom|accept|sendto' | tail -40
cat /tmp/exp4.kstack
```

The stack sampler runs in its own terminal because T4 is occupied by `ktrace` for the duration of the run. Sampling every 0.5 s across the whole experiment is what establishes that the thread is in the kqueue sleep path *continuously*, not merely at one convenient instant.

Screenshots


```

[root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # SX_VERBOSE=1 python3 -u experiment.py 4 2>&1 | tee /tmp/exp4.log
=== Experiment 4: One Client Should Not Stall the Others ===

Started Exchange Server (PID 45949).
[server] listening on 127.0.0.1:5000 (fd=3, kq=4)
[server] accepted fd=5 owner=1 from 127.0.0.1:25937
Exchange Server is listening.

[server] fd=5: connection reset by peer (RST, ECONNRESET)
[server] closing fd=5 owner=1
Client 1: connected from 127.0.0.1:33204

Client 1 will send an incomplete application message.
[server] accepted fd=5 owner=2 from 127.0.0.1:33204
Client 1 will now remain silent.

[recv] fd=5 n=20 data=LOGIN blocked_client
Client 2: connected from 127.0.0.1:38298[server] accepted fd=6 owner=3 from 127.0.0.1:38298

Client 2 will send a complete application message.
[recv] fd=6 n=20 data=LOGIN active_client

Client 2 response: 'OK'
Elapsed time: 0.001 seconds

Investigate the behavior of both TCP connections.
Press Ctrl-C when you are finished.
^C
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #

```

```

=== 06:51:12 ===

```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.33204	ESTABLISHED
tcp4	0	20	127.0.0.1.33204	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	*.*	LISTEN

```

=== 06:51:12 ===

```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.33204	ESTABLISHED
tcp4	0	0	127.0.0.1.33204	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	*.*	LISTEN

```

=== 06:51:12 ===

```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.33204	ESTABLISHED
tcp4	0	0	127.0.0.1.33204	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	*.*	LISTEN

```

=== 06:51:12 ===

```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.33204	ESTABLISHED
tcp4	0	0	127.0.0.1.33204	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	*.*	LISTEN

```

=== 06:51:12 ===

```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.33204	ESTABLISHED
tcp4	0	0	127.0.0.1.33204	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	*.*	LISTEN

```

=== 06:51:13 ===

```

```
Active Internet connections (including servers)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)
tcp4	0	0	127.0.0.1.5000	127.0.0.1.33204	ESTABLISHED


```

root@exchange:/home/cooluser1 # kdump -f /tmp/exp4.kt -T | egrep 'kevent|recvfrom|accept|sendto' | tail -
45949 exchange_server 1788916872.010460357 RET kevent 1
45949 exchange_server 1788916872.010464023 CALL accept(0x3,0x808b4628,0x808b4624)
45949 exchange_server 1788916872.010468190 RET accept 5
45949 exchange_server 1788916872.010474523 CALL kevent(0x4,0x808b4638,0x1,0,0,0)
45949 exchange_server 1788916872.010478315 RET kevent 0
45949 exchange_server 1788916872.010478898 CALL kevent(0x4,0x808b4638,0x1,0,0,0)
45949 exchange_server 1788916872.010479690 RET kevent 0
45949 exchange_server 1788916872.010546232 CALL accept(0x3,0x808b4628,0x808b4624)
45949 exchange_server 1788916872.010547982 RET accept -1 errno 35 Resource temporarily unavailable
45949 exchange_server 1788916872.010548940 CALL kevent(0x4,0,0,0x808b46e0,0x40,0)
45949 exchange_server 1788916872.010581773 RET kevent 1
45949 exchange_server 1788916872.010583607 CALL recvfrom(0x5,0x808b3658,0x1000,0,0,0)
45949 exchange_server 1788916872.010586898 RET recvfrom 20/0x14
45949 exchange_server 1788916872.010623982 CALL recvfrom(0x5,0x808b3658,0x1000,0,0,0)
45949 exchange_server 1788916872.010625315 RET recvfrom -1 errno 35 Resource temporarily unavailable
45949 exchange_server 1788916872.010626065 CALL kevent(0x4,0,0,0x808b46e0,0x40,0)
45949 exchange_server 1788916874.018004232 RET kevent 1
45949 exchange_server 1788916874.018021273 CALL accept(0x3,0x808b4628,0x808b4624)
45949 exchange_server 1788916874.018029482 RET accept 6
45949 exchange_server 1788916874.018115190 CALL kevent(0x4,0x808b4638,0x1,0,0,0)
45949 exchange_server 1788916874.018125732 RET kevent 0
45949 exchange_server 1788916874.018127232 CALL kevent(0x4,0x808b4638,0x1,0,0,0)
45949 exchange_server 1788916874.018129607 RET kevent 0
45949 exchange_server 1788916874.018145482 CALL accept(0x3,0x808b4628,0x808b4624)
45949 exchange_server 1788916874.018148690 RET accept -1 errno 35 Resource temporarily unavailable
45949 exchange_server 1788916874.018150357 CALL kevent(0x4,0,0,0x808b46e0,0x40,0)
45949 exchange_server 1788916874.018311732 RET kevent 1
45949 exchange_server 1788916874.018318107 CALL recvfrom(0x6,0x808b3658,0x1000,0,0,0)
45949 exchange_server 1788916874.018323690 RET recvfrom 20/0x14
45949 exchange_server 1788916874.018465982 CALL sendto(0x6,0x2388f809101,0x3,0,0,0)
45949 exchange_server 1788916874.019150607 RET sendto 3
45949 exchange_server 1788916874.019163982 CALL recvfrom(0x6,0x808b3658,0x1000,0,0,0)
45949 exchange_server 1788916874.019167148 RET recvfrom -1 errno 35 Resource temporarily unavailable
45949 exchange_server 1788916874.019169982 CALL kevent(0x4,0,0,0x808b46e0,0x40,0)
45949 exchange_server 1788916882.652881148 RET kevent 2
45949 exchange_server 1788916882.652904857 CALL recvfrom(0x5,0x808b3658,0x1000,0,0,0)
45949 exchange_server 1788916882.652908065 RET recvfrom 0
45949 exchange_server 1788916882.652958565 CALL recvfrom(0x6,0x808b3658,0x1000,0,0,0)
45949 exchange_server 1788916882.652991773 RET recvfrom 0
45949 exchange_server 1788916882.653048815 CALL kevent(0x4,0,0,0x808b46e0,0x40,0)

```

```

root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # cat /tmp/exp4_packets.txt
2026-09-09 06:51:11.908476 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3cba))
127.0.0.1.21680 > 127.0.0.1.5000: Flags [S], cksum 0xfe30 (incorrect -> 0x4bb3), seq 96982448, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3525881430 ecr 0], length 0
2026-09-09 06:51:11.908492 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 40, bad cksum 0 (->3cba))
127.0.0.1.5000 > 127.0.0.1.21680: Flags [R.], cksum 0xfefc (incorrect -> 0x0e25), seq 0, ack 96982448, win 0, length 0
2026-09-09 06:51:12.009885 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3cba))
127.0.0.1.25937 > 127.0.0.1.5000: Flags [S], cksum 0xfe30 (incorrect -> 0x080e), seq 104743647, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 2365962448 ecr 0], length 0
2026-09-09 06:51:12.009677 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3cba))
127.0.0.1.5000 > 127.0.0.1.25937: Flags [S.], cksum 0xfe30 (incorrect -> 0x7713), seq 2406888824, ack 104743648, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 4127446518 ecr 2365962448], length 0
2026-09-09 06:51:12.009709 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
127.0.0.1.25937 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0xbdb5), seq 104743648, ack 2406888825, win 1277, options [nop,nop,TS val 2365962448 ecr 4127446518], length 0
2026-09-09 06:51:12.009896 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 52, bad cksum 0 (->3cc2))
127.0.0.1.25937 > 127.0.0.1.5000: Flags [R.], cksum 0xfe28 (incorrect -> 0xdffe), seq 104743648, ack 2406888825, win 0, options [nop,nop,TS val 2365962448 ecr 4127446518], length 0
2026-09-09 06:51:12.010154 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3cba))
127.0.0.1.33204 > 127.0.0.1.5000: Flags [S], cksum 0xfe30 (incorrect -> 0x8e27), seq 3626259302, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3787317037 ecr 0], length 0
2026-09-09 06:51:12.010165 IP (tos 0x0, ttl 64, id 0, offset 0, flags [DF], proto TCP (6), length 60, bad cksum 0 (->3cba))
127.0.0.1.5000 > 127.0.0.1.33204: Flags [S.], cksum 0xfe30 (incorrect -> 0xdaa8), seq 3419622716, ack 3626259303, win 65535, options [mss 16344,nop,wscale 6,sackOK,TS val 3959598682 ecr 3787317037], length 0

```

```

root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # cat /tmp/exp4.kstack
06:51:12
PID TID COMM TDNAME KSTACK
45949 100191 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent kern_kevent_generic sys_kevent do_eio_sync handle_eio_sync
06:51:13
PID TID COMM TDNAME KSTACK
45949 100191 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent kern_kevent_generic sys_kevent do_eio_sync handle_eio_sync
06:51:13
PID TID COMM TDNAME KSTACK
45949 100191 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent kern_kevent_generic sys_kevent do_eio_sync handle_eio_sync
06:51:14
PID TID COMM TDNAME KSTACK
45949 100191 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent kern_kevent_generic sys_kevent do_eio_sync handle_eio_sync
06:51:14
PID TID COMM TDNAME KSTACK
45949 100191 exchange_server - mi_switch sleepq_catch_signals sleepq_wait_sig _sleep kqueue_scan kqueue_kevent kern_kevent kern_kevent_generic sys_kevent do_eio_sync handle_eio_sync
06:51:15

```

Observation

1. **Latency.** The harness reports Client 2's **OK** arriving in well under a millisecond, despite Client 1 being connected and mid-message throughout.
2. **Syscall timeline (**kdump**).** After reading Client 1's 20 partial bytes, **recvfrom()** returns **EAGAIN** and the server enters **kevent()**. Two seconds later **kevent()** returns for the listening descriptor, the server accepts Client 2, reads its complete **LOGIN**, and replies.

3. **Kernel stack (`procstat -k`)**. Every sample, both before and after Client 2 arrives, ends in the kqueue sleep path:

```
mi_switch sleepq_catch_signals sleepq_wait_sig _sleep
kqueue_scan kqueue_kevent kern_kevent kern_kevent_generic sys_kevent
do_el0_sync handle_el0_sync
```

The diagnostic point is what is **absent**: there is no `sbwait` and no `soreceive_generic` anywhere in the stack. A server blocked in `recv()` on a specific socket would show exactly those frames.

4. **Kernel queues**. Client 1's connection shows `Recv-Q 0, Send-Q 0, ESTABLISHED` — identical to Client 2's. At the TCP layer the idle connection is indistinguishable from any other; its incompleteness exists solely in the server's userspace `LineBuffer`.
5. **Wire**. After Client 1's 20-byte segment and its ACK, no further traffic appears on that 4-tuple. Client 2's SYN, data, and reply all complete within the same millisecond.

Answer

Yes. The server accepts and services Client 2 with sub-millisecond latency while Client 1 remains connected and silent.

The operation that determines this is the **readiness-notification call**: `kevent()` with a NULL timeout, which blocks on the set of registered descriptors and returns only those that are actually ready. `procstat -k` shows the single server thread asleep in `kqueue_scan` / `kern_kevent` for the entire run and never in the socket-receive path. Client 1's socket simply never becomes readable, so it never appears in the returned event list and costs nothing; its 20 partial bytes were already copied into userspace and wait there as an incomplete line.

Had the server instead issued a blocking `recv()` on Client 1's descriptor — or served connections sequentially — the thread would have slept in `sbwait` inside that one socket's receive path, and Client 2's `LOGIN` would have gone unanswered until Client 1 sent its newline. Non-blocking descriptors combined with readiness notification are what decouple the server's progress from any individual client's behaviour.

5. Experiment 5 — Multiple Clients and I/O Multiplexing (optional)

Question. At a particular point during the experiment, which client connections are actually ready for the server to service, and what evidence from the running system allows you to determine this?

Approach

Five simultaneous connections were established. Clients 1, 3 and 5 each sent a 16-byte `LOGIN client_N\n` at 0.5 s intervals; Clients 2 and 4 remained completely silent.

Readiness in this experiment is *transient*: the server drains each socket within microseconds of it becoming readable, so polling `Recv=0` almost always returns 0 and cannot by itself identify which sockets were ready. The rigorous evidence is therefore the **syscall trace**, which records which descriptors the event loop actually acted on, cross-referenced against an fd↔ephemeral-port map built from `procstat -f` and `sockstat` and the harness's printed source ports.

Commands and tools

```
# T4 – attach the syscall trace as the server spawns
while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done
ktrace -i -f /tmp/exp5.kt -p $PID -t c

# T2 – extended poller, for rcvtime and socket-buffer state across all five
connections
~/tools/poll-tcpv.sh 0.1 /tmp/exp5.tcpx

# T3 – packet capture
tcpdump -l -i lo0 -n -S -tttt 'tcp port 5000' > /tmp/exp5_packets.txt

# T1 – harness
SX_VERBOSE=1 python3 -u experiment.py 5 2>&1 | tee /tmp/exp5.log

# T5 – fd mapping, captured DURING the 15 s observation window, before the
server closes
PID=$(pgrep -n exchange_server)
procstat -f $PID | grep TCP | tee /tmp/exp5.fdmapp
sockstat -4 -c -p 5000

# T4 – post-run extraction
ktrace -C
kdump -f /tmp/exp5.kt -T | egrep 'kevent|recvfrom' | tee /tmp/exp5.syscalls
kdump -f /tmp/exp5.kt -T | grep recvfrom \
  | awk '{print $NF, $0}' | grep -o 'recvfrom(0x[0-9a-f]*)' | sort | uniq -c
```

The fd map must be taken while the connections are live: once the server exits, `procstat -f` has nothing to report and the descriptor-to-port correspondence is lost.

Screenshots


```

0 0 127.0.0.1.5000 *.* 0 0 0 0 0 0 0 0 0.00 0.00 0.00 0.00 1.00
=== 08:12:30 ===
Active Internet connections (including servers)
Proto Recv-Q Send-Q Local Address Foreign Address R-HIWA S-HIWA R-LOWA S-LOWA R-BCNT S-BCNT R-BMAX S-BMAX rexmt persist keep 2msl delack rcvtime
tcp4 0 0 127.0.0.1.5000 127.0.0.1.59662 81720 49832 1 2848 0 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 0.00
tcp4 3 0 127.0.0.1.59662 127.0.0.1.5000 81720 49832 1 2848 256 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 0.00
tcp4 0 0 127.0.0.1.5000 127.0.0.1.35120 81720 49832 1 2848 0 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 1.00
tcp4 0 0 127.0.0.1.35120 127.0.0.1.5000 81720 49832 1 2848 0 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 1.00
tcp4 0 0 127.0.0.1.5000 127.0.0.1.37828 81720 49832 1 2848 0 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 0.50
tcp4 3 0 127.0.0.1.37828 127.0.0.1.5000 81720 49832 1 2848 256 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 0.50
tcp4 0 0 127.0.0.1.35050 127.0.0.1.35050 81720 49832 1 2848 0 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 1.00
tcp4 0 0 127.0.0.1.35050 127.0.0.1.5000 81720 49832 1 2848 0 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 1.00
tcp4 0 0 127.0.0.1.5000 127.0.0.1.53560 81720 49832 1 2848 0 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 1.00
tcp4 3 0 127.0.0.1.53560 127.0.0.1.5000 81720 49832 1 2848 256 0 653760 392256 0.00 0.00 7198.92 0.00 0.00 1.00
tcp4 0 0 127.0.0.1.5000 *.*.* 0 0 0 0 0 0 0 0 0.00 0.00 0.00 0.00 1.10
=== 08:12:30 ===
Active Internet connections (including servers)

```



```
[root@exchange:/home/cooluser1 # kdump -f /tmp/exp5.kt -T | egrep 'kevent|recvfrom' | tee /tmp/exp5.sysca
48312 exchange_server 1788921749.808713937 RET kevent 1
48312 exchange_server 1788921749.808767437 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.808771770 RET kevent 0
48312 exchange_server 1788921749.808772520 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.808773770 RET kevent 0
48312 exchange_server 1788921749.808806020 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921749.808807437 RET kevent 1
48312 exchange_server 1788921749.808825353 CALL recvfrom(0x5,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921749.808826937 RET recvfrom -1 errno 54 Connection reset by peer
48312 exchange_server 1788921749.808861103 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921749.808930228 RET kevent 1
48312 exchange_server 1788921749.808938978 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.808940228 RET kevent 0
48312 exchange_server 1788921749.808940978 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.808941728 RET kevent 0
48312 exchange_server 1788921749.808959603 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921749.808973770 RET kevent 1
48312 exchange_server 1788921749.808980312 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.808981478 RET kevent 0
48312 exchange_server 1788921749.808982187 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.808982978 RET kevent 0
48312 exchange_server 1788921749.808991770 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921749.809109103 RET kevent 1
48312 exchange_server 1788921749.809130562 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.809132645 RET kevent 0
48312 exchange_server 1788921749.809133520 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.809134562 RET kevent 0
48312 exchange_server 1788921749.809391728 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.809393103 RET kevent 0
48312 exchange_server 1788921749.809393853 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.809394645 RET kevent 0
48312 exchange_server 1788921749.809598687 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.809600270 RET kevent 0
48312 exchange_server 1788921749.809601145 CALL kevent(0x4,0x80c06f88,0x1,0,0,0)
48312 exchange_server 1788921749.809602020 RET kevent 0
48312 exchange_server 1788921749.809613895 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921749.809614978 RET kevent 1
48312 exchange_server 1788921749.809616228 CALL recvfrom(0x5,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921749.809618478 RET recvfrom 15/0xf
48312 exchange_server 1788921749.809779228 CALL recvfrom(0x5,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921749.809780187 RET recvfrom -1 errno 35 Resource temporarily unavailable
48312 exchange_server 1788921749.809781478 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921750.332646812 RET kevent 1
48312 exchange_server 1788921750.332672603 CALL recvfrom(0x7,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921750.332684478 RET recvfrom 15/0xf
48312 exchange_server 1788921750.332770645 CALL recvfrom(0x7,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921750.332772728 RET recvfrom -1 errno 35 Resource temporarily unavailable
48312 exchange_server 1788921750.332775228 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921750.843996020 RET kevent 1
48312 exchange_server 1788921750.844029020 CALL recvfrom(0x9,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921750.844054270 RET recvfrom 15/0xf
48312 exchange_server 1788921750.844112145 CALL recvfrom(0x9,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921750.844113145 RET recvfrom -1 errno 35 Resource temporarily unavailable
48312 exchange_server 1788921750.844114228 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
48312 exchange_server 1788921761.295811228 RET kevent 5
48312 exchange_server 1788921761.295834562 CALL recvfrom(0x5,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921761.295870228 RET recvfrom 0
48312 exchange_server 1788921761.295981687 CALL recvfrom(0x6,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921761.295983895 RET recvfrom 0
48312 exchange_server 1788921761.296012770 CALL recvfrom(0x7,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921761.296014562 RET recvfrom 0
48312 exchange_server 1788921761.296038270 CALL recvfrom(0x8,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921761.296039937 RET recvfrom 0
48312 exchange_server 1788921761.296062312 CALL recvfrom(0x9,0x80c05fa8,0x1000,0,0,0)
48312 exchange_server 1788921761.296063895 RET recvfrom 0
48312 exchange_server 1788921761.296132853 CALL kevent(0x4,0,0,0x80c07030,0x40,0)
```

```
[root@exchange:/home/cooluser1 # kdump -f /tmp/exp5.kt -T | grep recvfrom | awk '{print $NF, $0}' | grep -o 'recvfrom(0x[0-9a-f]*)' | sort | uniq -c
8 recvfrom(0x5
2 recvfrom(0x6
6 recvfrom(0x7
2 recvfrom(0x8
6 recvfrom(0x9
root@exchange:/home/cooluser1 #
```

```

root@exchange:/home/cooluser1 # netstat -an -p tcp -x
Active Internet connections (including servers)
Proto Recv-Q Send-Q Local Address Foreign Address R-HIWA S-HIWA R-LOWA S-LOWA R-BCNT S-BCNT R-BMAX S-BMAX rexmt persist keep 2msl delack rcvtime
tcp4 0 0 127.0.0.1.5000 *.* 65700 33500 1 2048 0 0 0 0 0 0 0.00 0.00 0.00 0.00 0.00 1215.50
tcp4 0 0 10.0.2.15.22 10.0.2.2.54775 65700 33500 1 2048 0 0 0 0 0 0 0.00 0.00 223.24 0.00 0.00 0.00
tcp4 0 0 10.0.2.15.22 10.0.2.2.54772 65700 33500 1 2048 0 0 0 0 0 0 0.00 0.00 167.48 0.00 0.00 124.06
tcp4 0 0 10.0.2.15.22 10.0.2.2.49667 65700 33500 1 2048 0 0 0 0 0 0 0.00 0.00 1146.42 0.00 0.00 126.03
tcp4 0 0 10.0.2.15.22 10.0.2.2.61661 65700 33500 1 2048 0 0 0 0 0 0 0.00 0.00 1119.32 0.00 0.00 1201.95
tcp4 0 0 10.0.2.15.22 10.0.2.2.60334 65700 33500 1 2048 0 0 0 0 0 0 0.00 0.00 1130.73 0.00 0.00 1195.93
tcp4 0 0 *.22 *.* 0 0 0 0 0 0 0 0 0 0 0.00 0.00 0.00 0.00 0.00 30169.67
tcp6 0 0 *.22 *.* 0 0 0 0 0 0 0 0 0 0 0.00 0.00 0.00 0.00 0.00 30169.67
root@exchange:/home/cooluser1 #

```

Observation

The client↔descriptor mapping was established from `sockstat` and the ephemeral ports printed by the harness: descriptors 5, 7 and 9 belong to the active clients and 6 and 8 to the silent ones. The per-descriptor syscall counts are decisive:

```

8 recvfrom(0x5 ← Client 1: active (LOGIN read + EAGAIN + teardown)
2 recvfrom(0x6 ← Client 2: idle (teardown EOF reads only)
6 recvfrom(0x7 ← Client 3: active
2 recvfrom(0x8 ← Client 4: idle (teardown EOF reads only)
6 recvfrom(0x9 ← Client 5: active

```

Descriptors 6 and 8 received exactly two `recvfrom` calls each, both returning 0 (EOF), and both occurring only during the final teardown when the harness closed every connection. **During the observation phase, `kevent()` never once woke the server for those descriptors.**

`procstat -f` additionally shows all five connected descriptors held simultaneously by a single thread, alongside fd 3 (listener) and fd 4 (kqueue).

Answer

During the active phase, the connections ready for service were exactly those of **Clients 1, 3 and 5**, corresponding to descriptors 5, 7 and 9. `kevent()` returned each individually as its data arrived and never returned descriptors 6 or 8, because those sockets never became readable. At the later moment of static inspection, no connection was ready at all — every arrived byte had already been consumed.

The determining evidence is the syscall trace: the server issued `recvfrom()` only on the ready descriptors, and never syscalled on the idle ones despite both being registered with `EVFILT_READ`. This demonstrates the central property of readiness-based multiplexing: the kernel evaluates readiness on the server's behalf and returns only the subset that can make progress, so the cost of an idle connection is one kernel `knote` and a socket buffer — not a thread, not a stack, and not a single wasted syscall.

6. Experiment 6 — FIN vs. RST: Orderly and Abrupt Connection Termination

Question. How does an abrupt client termination differ from the orderly shutdown? Identify the TCP event observed on the network and the corresponding behaviour of the Exchange Server's socket.

Approach

The harness performs both terminations in a single run, which allows them to be compared on one continuous packet timeline. Part A closes orderly via `shutdown(SHUT_WR)`; Part B closes abortively by setting `SO_LINGER{1, 0}` before `close()`, which forces an RST. Both the wire and the server's syscall return values were captured. The startup probe RST described in §0.5 was excluded.

Commands and tools

```
# T4 – attach the syscall trace as the server spawns
while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done
ktrace -i -f /tmp/exp6.kt -p $PID -t c

# T2 – state poller
~/tools/poll-tcp.sh 0.1 /tmp/exp6.states

# T3 – packet capture
tcpdump -l -i lo0 -n -S -vv -tttt 'tcp port 5000' > /tmp/exp6_packets.txt

# T1 – harness
SX_VERBOSE=1 python3 -u experiment.py 6 2>&1 | tee /tmp/exp6.log

# T5 – run TWICE, once per phase.
# (a) during Part A's 10 s window, after "orderly connection termination"
# (b) immediately after "The client will now close abortively."
PID=$(pgrep -n exchange_server)
netstat -an -p tcp | egrep 'Proto|\.5000'
procstat -f $PID | grep TCP

# T4 – post-run extraction
ktrace -C
kdump -f /tmp/exp6.kt -T | egrep 'kevent|recvfrom|shutdown|close' | tail
-40
grep -E 'peer closed|reset by peer' /tmp/exp6.log
cat /tmp/exp6_packets.txt | egrep 'Flags \[F|Flags \[R'
```

The two T5 snapshots are the core of this experiment: the first captures the client in `TIME_WAIT` after the orderly close, the second captures both endpoints already absent after the reset.

Screenshots


```
[root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # SX_VERBOSE=1 python3 -u experiment.py 6 2>&1 | tee /tmp/exp6.log
=== Experiment 6: FIN vs. RST: Orderly and Abrupt Connection Termination ===
```

```
Started Exchange Server (PID 52812).
```

```
[server] listening on 127.0.0.1:5000 (fd=3, kq=4)
```

```
Exchange Server is listening.[server] accepted fd=5 owner=1 from 127.0.0.1:12772
```

```
Part A: orderly connection termination (FIN).
```

```
[server] fd=5: connection reset by peer (RST, ECONNRESET)
```

```
[server] closing fd=5 owner=1
```

```
[server] accepted fd=5 owner=2 from 127.0.0.1:34527
```

```
Orderly client: connected from 127.0.0.1:34527
```

```
The client will shut down its sending direction.
```

```
[server] fd=5: peer closed (FIN, recv==0)
```

```
[server] closing fd=5 owner=2
```

```
Investigate the TCP traffic and connection state.
```

```
Part B: abortive connection termination (RST).
```

```
Abrupt client: connected from 127.0.0.1:14618
```

```
[server] accepted fd=5 owner=3 from 127.0.0.1:14618
```

```
The client will now close abortively.
```

```
Investigate the TCP traffic and connection state.
```

```
[server] fd=5: connection reset by peer (RST, ECONNRESET)
```

```
The Exchange Server will remain running for 15 seconds.
```

```
[server] closing fd=5 owner=3
```

```
Experiment finished.
```

```
Stopping Exchange Server...
```

```
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # █
```

```
52812 exchange_server 1788923376.571039935 RET kevent -1 errno 4 Interrupted system call
```

```
[root@exchange:/home/cooluser1 # grep -E 'peer closed|reset by peer' /tmp/exp6.log
```

```
[server] fd=5: connection reset by peer (RST, ECONNRESET)
```

```
[server] fd=5: peer closed (FIN, recv==0)
```

```
[server] fd=5: connection reset by peer (RST, ECONNRESET)
```

```
[root@exchange:/home/cooluser1 # cat /tmp/exp6_packets.txt | egrep 'Flags \[F|Flags \[R'
127.0.0.1.5000 > 127.0.0.1.13630: Flags [R.], cksum 0xfe1c (incorrect -> 0x8333), seq 0, ack 33698
```

```
[root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # PID=$(pgrep -n exchange_server); netstat -an -p tcp | egrep 'Proto|\.5000'; procstat -f $PID | grep TCP
Proto Recv-Q Send-Q Local Address Foreign Address (state)
tcp4 0 0 127.0.0.1.34527 127.0.0.1.5000 TIME_WAIT
tcp4 0 0 127.0.0.1.5000 *.* LISTEN
52812 exchange_server 3 s - rw---n-- 2 0 TCP 0 0 127.0.0.1:5000 *:0
[root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # PID=$(pgrep -n exchange_server); netstat -an -p tcp | egrep 'Proto|\.5000'; procstat -f $PID | grep TCP
Proto Recv-Q Send-Q Local Address Foreign Address (state)
tcp4 0 0 127.0.0.1.34527 127.0.0.1.5000 TIME_WAIT
tcp4 0 0 127.0.0.1.5000 *.* LISTEN
52812 exchange_server 3 s - rw---n-- 2 0 TCP 0 0 127.0.0.1:5000 *:0
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # █
```

```

root@exchange:/home/cooluser1 # while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done; ktrace -i -f /tmp/exp6.log
root@exchange:/home/cooluser1 # ktrace -C
root@exchange:/home/cooluser1 # kdump -f /tmp/exp6.log -T | egrep 'kevent|recvfrom|shutdown|close' | tail -40
52812 exchange_server 1788923351.527012560 CALL kevent(0x4,0x810a45b8,0x1,0,0,0)
52812 exchange_server 1788923351.527013519 RET kevent 0
52812 exchange_server 1788923351.527036394 CALL kevent(0x4,0,0,0x810a4660,0x40,0)
52812 exchange_server 1788923351.527037394 RET kevent 1
52812 exchange_server 1788923351.527059227 CALL recvfrom(0x5,0x810a35d8,0x1000,0,0,0)
52812 exchange_server 1788923351.527061644 RET recvfrom -1 errno 54 Connection reset by peer
52812 exchange_server 1788923351.527072060 CALL shutdown(0x5,SHUT_WR)
52812 exchange_server 1788923351.527072644 RET shutdown -1 errno 57 Socket is not connected
52812 exchange_server 1788923351.527080019 CALL close(0x5)
52812 exchange_server 1788923351.527085935 RET close 0
52812 exchange_server 1788923351.527095477 CALL kevent(0x4,0,0,0x810a4660,0x40,0)
52812 exchange_server 1788923351.527161602 RET kevent 1
52812 exchange_server 1788923351.527169685 CALL kevent(0x4,0x810a45b8,0x1,0,0,0)
52812 exchange_server 1788923351.527171102 RET kevent 0
52812 exchange_server 1788923351.527171644 CALL kevent(0x4,0x810a45b8,0x1,0,0,0)
52812 exchange_server 1788923351.527172185 RET kevent 0
52812 exchange_server 1788923351.527187560 CALL kevent(0x4,0,0,0x810a4660,0x40,0)
52812 exchange_server 1788923351.527204477 RET kevent 1
52812 exchange_server 1788923351.527206019 CALL recvfrom(0x5,0x810a35d8,0x1000,0,0,0)
52812 exchange_server 1788923351.527206852 RET recvfrom 0
52812 exchange_server 1788923351.527213602 CALL shutdown(0x5,SHUT_WR)
52812 exchange_server 1788923351.527227769 RET shutdown 0
52812 exchange_server 1788923351.527235894 CALL close(0x5)
52812 exchange_server 1788923351.527238310 RET close 0
52812 exchange_server 1788923351.527239727 CALL kevent(0x4,0,0,0x810a4660,0x40,0)
52812 exchange_server 1788923361.551508102 RET kevent 1
52812 exchange_server 1788923361.551546602 CALL kevent(0x4,0x810a45b8,0x1,0,0,0)
52812 exchange_server 1788923361.551552810 RET kevent 0
52812 exchange_server 1788923361.551553935 CALL kevent(0x4,0x810a45b8,0x1,0,0,0)
52812 exchange_server 1788923361.551555144 RET kevent 0
52812 exchange_server 1788923361.551590602 CALL kevent(0x4,0,0,0x810a4660,0x40,0)
52812 exchange_server 1788923361.551669310 RET kevent 1
52812 exchange_server 1788923361.551672435 CALL recvfrom(0x5,0x810a35d8,0x1000,0,0,0)
52812 exchange_server 1788923361.551675644 RET recvfrom -1 errno 54 Connection reset by peer
52812 exchange_server 1788923361.551682310 CALL shutdown(0x5,SHUT_WR)
52812 exchange_server 1788923361.551683060 RET shutdown -1 errno 57 Socket is not connected
52812 exchange_server 1788923361.551694310 CALL close(0x5)
52812 exchange_server 1788923361.551706019 RET close 0
52812 exchange_server 1788923361.551714977 CALL kevent(0x4,0,0,0x810a4660,0x40,0)
52812 exchange_server 1788923376.571039935 RET kevent -1 errno 4 Interrupted system call
root@exchange:/home/cooluser1 # grep -E 'peer closed|reset by peer' /tmp/exp6.log
[cooluser1 fd 5] connection reset by peer (RST, ECONNRESET)

```

```

root@exchange:/home/cooluser1 # cat /tmp/exp6.packets.txt | egrep 'Flags \[F\]Flags \[R\]
127.0.0.1.5000 > 127.0.0.1.13630: Flags [R.], cksum 0xfe28 (incorrect -> 0x8333), seq 0, ack 3369868536, win 0, length 0
127.0.0.1.12772 > 127.0.0.1.5000: Flags [R.], cksum 0xfe28 (incorrect -> 0x9245), seq 2039256153, ack 3824572438, win 0, options [nop,nop,TS val 2183201508 ecr 3745968837], length 0
127.0.0.1.34527 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0x3e77), seq 11967538, ack 742257371, win 1277, options [nop,nop,TS val 3883613547 ecr 89633950], length 0
127.0.0.1.5000 > 127.0.0.1.34527: Flags [F.], cksum 0xfe28 (incorrect -> 0x3e76), seq 742257371, ack 11967539, win 1277, options [nop,nop,TS val 89633950 ecr 3883613547], length 0
127.0.0.1.14618 > 127.0.0.1.5000: Flags [R.], cksum 0xfe28 (incorrect -> 0x12a6), seq 3824886053, ack 2023439248, win 0, options [nop,nop,TS val 813734190 ecr 712010755], length 0
root@exchange:/home/cooluser1 #

```

Observation

	Part A — orderly	Part B — abrupt
Trigger	<code>shutdown(fd, SHUT_WR)</code>	<code>SO_LINGER{1,0}</code> then <code>close()</code>
Packets on the wire	FIN handshake: <code>[F.]</code> → <code>[.]</code> → <code>[F.]</code> → <code>[.]</code>	a single <code>[R.]</code> , no FIN at all
Sequence semantics	FIN consumes one sequence number and is ACKed	RST carries none and is never ACKed
Queued data	delivered before the close completes	discarded in both socket buffers
Server syscall result	<code>recvfrom()</code> returns <code>0</code>	<code>recvfrom()</code> returns <code>-1, errno 54 (ECONNRESET)</code>
Server log line	<code>peer closed (FIN, recv==0)</code>	<code>connection reset by peer (RST, ECONNRESET)</code>

	Part A — orderly	Part B — abrupt
Post-teardown state	client endpoint enters TIME_WAIT	both endpoints vanish immediately; no TIME_WAIT

In Part A the client only half-closes, but the server responds to EOF by closing its own side immediately, so the connection completes teardown in both directions and the client transitions **FIN_WAIT_2** → **TIME_WAIT** on receiving the server's FIN.

Answer

The TCP event. Orderly termination is a *negotiated, in-band* end of stream: FIN occupies a sequence number, is acknowledged, and is delivered to the peer application as **recv() == 0**. Data already queued is still delivered, each direction closes independently, and the initiator holds **TIME_WAIT** for $2 \times \text{MSL}$ so that a delayed duplicate segment cannot corrupt a later connection reusing the same 4-tuple. Abrupt termination is a single *out-of-band* RST: it carries no sequence semantics, is never acknowledged, discards queued data at both ends, tears down both PCBs immediately, and leaves no **TIME_WAIT**.

The server socket's behaviour. **kevent()** reports the descriptor readable in both cases — the difference appears only at **recv()**. An orderly close yields a return of 0, meaning end of stream; a reset yields -1 with **ECONNRESET** (errno 54). The server distinguishes these two branches explicitly and logs them with different wording, then routes both into the same **close_conn()** path. Writing to a reset connection would additionally raise **EPIPE**, which is harmless here because **SIGPIPE** is ignored at startup.

7. Experiment 7 — Backpressure and the Slow Receiver

Question. How does the Exchange Server's TCP connection to the slow client behave as the client stops reading, and what evidence shows whether this eventually affects the server's ability to communicate with other clients?

Approach

The harness connects two Market-Data Clients subscribed to JNST and two Trader Clients, then generates 5000 matching BUY/SELL pairs at roughly 1 ms intervals. One subscriber is drained continuously; the other never reads a single byte. Each trade emits **TRADE JNST 1 238** = 17 bytes to each subscriber, so the slow socket must absorb $5000 \times 17 = \mathbf{85,000 \text{ bytes}}$.

Four independent evidence streams were captured simultaneously: the wire (zero-window advertisements and persist probes), the kernel queues (**Recv-Q/Send-Q** and the extended socket state), the application (the exact instant **send()** refuses), and the liveness of the other three clients.

A necessary control: why the first run showed nothing

The experiment was first run against stock tunables and produced **no backpressure at all**. The maximum queue analysis showed:

127.0.0.1.<slow>	maxRecvQ=85000	maxSendQ=0
127.0.0.1.5000	maxRecvQ=0	maxSendQ=108

`maxRecvQ = 85000` is exactly the whole trade stream. With FreeBSD's receive-buffer auto-tuning enabled, the slow client's socket buffer simply grew to absorb the entire burst: the window never closed, `send()` never refused, and no `[flush]` line was ever emitted. This is a genuine and instructive result — **at this data volume, default FreeBSD buffers absorb the whole stream and flow control never engages** — and it is reported here as the control case.

To make the mechanism observable, the socket buffers were clamped. Note that `net.inet.tcp.recvspace` alone is insufficient, since it only sets the *initial* size; `kern.ipc.maxsockbuf` is the hard ceiling the kernel enforces against every socket buffer:

```
sysctl kern.ipc.maxsockbuf          # recorded, then restored
afterwards
sysctl kern.ipc.maxsockbuf=16384
sysctl net.inet.tcp.recvspace=4096  net.inet.tcp.sendspace=4096
sysctl net.inet.tcp.recvbuf_auto=0  net.inet.tcp.sendbuf_auto=0
```

All tunables were restored to their defaults before Experiment 8.

Commands and tools

```
# T2 – state poller, filtering on both sides of port 5000
while ;; do
    printf '=== %s ===\n' "$(date '+%H:%M:%S')"
    netstat -an -p tcp | awk 'NR<=2 || $4 ~ /\.5000$/ || $5 ~ /\.5000$/'
    sleep 0.5
done | tee /tmp/exp7_states.log

# T3 – packet capture; -B 4096 enlarges the BPF buffer so the burst is not
dropped
tcpdump -i lo0 -n -S -vv -tttt -B 4096 'tcp port 5000' | tee /tmp/exp7.txt

# T4 – self-attaching monitor (see §0.6.2)
~/tools/watch7.sh | tee /tmp/exp7_t4.log

# T1 – harness
python3 -u experiment.py 7 2>&1 | tee /tmp/exp7.log

# T5 – run WHILE T1 is paused at "Press Ctrl-C when you are finished":
#       this is the only moment the persist timer and full buffers are
observable
netstat -an -p tcp -x | awk 'NR<=2 || /\.5000$/'
```

Post-run extraction:


```

grep -c '\[flush\]' /tmp/exp7.log # number of EAGAIN
refusals
grep '\[flush\]' /tmp/exp7.log | tail -3 # peak bytes queued in
userspace
grep -n 'win 0' /tmp/exp7.txt | grep -v 'Flags \[R' # zero-window ACKs,
excluding RSTs
awk '$1=="tcp4" { if ($2+0>mr[$4]) mr[$4]=$2+0; if ($3+0>ms[$4])
ms[$4]=$3+0 }
END { for (k in mr) printf "%-26s maxRecvQ=%-8d maxSendQ=%d\n", k,
mr[k], ms[k] }' \
/tmp/exp7_states.log | sort

```

The `grep -v 'Flags \[R'` filter is necessary because an RST also carries `win 0`; without it the harness's startup probe (\$0.5) is misread as a zero-window advertisement. A true zero-window advertisement is a pure ACK, `Flags \[.]`.

The `awk` reduction over the whole poller log, rather than reading its first screenful, is what distinguishes the two runs: it reports the maximum queue depth reached per endpoint across the entire experiment.

Screenshots

```

root@exchange:/home/vijeta # netstat -an -p tcp -x | awk 'NR<=2 || /\.5000/'
Active Internet connections (including servers)
Proto Recv-Q Send-Q Local Address Foreign Address R-HIWA S-HIWA R-LOWA S-LOWA R-BCNT S-BCNT R-BMAX S-BMAX rexmt persist keep 2msl delack rcvtime
tcp4 16 36 127.0.0.1.5000 127.0.0.1.49157 4096 4096 1 2048 256 256 16384 16384 0.25 0.00 7153.21 0.00 0.03 0.00
tcp4 36 16 127.0.0.1.49157 127.0.0.1.5000 4096 4096 1 2048 256 256 16384 16384 0.25 0.00 7153.21 0.00 0.03 0.00
tcp4 0 38 127.0.0.1.5000 127.0.0.1.13333 4096 4096 1 2048 0 256 16384 16384 0.28 0.00 7153.21 0.00 0.00 0.01
tcp4 0 0 127.0.0.1.13333 127.0.0.1.5000 4096 4096 1 2048 0 0 16384 16384 0.00 0.00 7153.21 0.00 0.02 0.01
tcp4 0 4081 127.0.0.1.5000 127.0.0.1.31823 4096 4096 1 2048 0 4864 16384 16384 0.00 12.31 7153.21 0.00 0.00 0.68
tcp4 4096 0 127.0.0.1.31823 127.0.0.1.5000 4096 4096 1 2048 12800 0 16384 16384 0.00 0.00 7153.21 0.00 0.00 0.68
tcp4 0 17 127.0.0.1.5000 127.0.0.1.48722 4096 4096 1 2048 0 256 16384 16384 0.25 0.00 7153.21 0.00 0.00 0.00
tcp4 17 0 127.0.0.1.48722 127.0.0.1.5000 4096 4096 1 2048 256 0 16384 16384 0.00 0.00 7153.21 0.00 0.03 0.00
tcp4 0 0 127.0.0.1.5000 *.* 0 0 0 0 0 0 0 0 0.00 0.00 0.00 0.00 0.00 46.88
root@exchange:/home/vijeta # netstat -an -p tcp -x | awk 'NR<=2 || /\.5000/'
Active Internet connections (including servers)
Proto Recv-Q Send-Q Local Address Foreign Address R-HIWA S-HIWA R-LOWA S-LOWA R-BCNT S-BCNT R-BMAX S-BMAX rexmt persist keep 2msl delack rcvtime
root@exchange:/home/vijeta #

```

```

root@exchange:/home/vijeta/COL-334-Assignment2- # python3 -u experiment.py 7 2>&1 | tee /tmp/exp7.log
=== Experiment 7: Backpressure and the Slow Receiver ===

```

```

Started Exchange Server (PID 24256).
[server] listening on 127.0.0.1:5000 (fd=3, kq=4)
[server] accepted fd=5 owner=1 from 127.0.0.1:18390
Exchange Server is listening.
[server] fd=5: connection reset by peer (RST, ECONNRESET)

[server] closing fd=5 owner=1
[server] accepted fd=5 owner=2 from 127.0.0.1:48722
Normal Market-Data Client: connected from 127.0.0.1:48722
Slow Market-Data Client: connected from 127.0.0.1:31823
Buyer Trader: connected from 127.0.0.1:13333
[server] accepted fd=6 owner=3 from 127.0.0.1:31823
[server] accepted fd=7 owner=4 from 127.0.0.1:13333
[server] accepted fd=8 owner=5 from 127.0.0.1:49157
Seller Trader: connected from 127.0.0.1:49157

```

```

Both Market-Data Clients are subscribed to JNST.
The normal client will keep reading.
The slow client will not read market-data updates.

```

```

Generating up to 5000 matching trades...

```

```

[flush] fd=6 send() EAGAIN, 17 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 34 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 51 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 68 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 85 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 102 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 119 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 136 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 153 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 170 bytes still queued, enabling EVFILT_WRITE

```

```
[flush] fd=6 send() EAGAIN, 7668/ bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76704 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76721 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76738 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76755 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76772 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76789 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76806 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 76823 bytes still queued, enabling EVFILT_WRITE
```

Generated 5000 matching trades.

The slow client has intentionally not consumed its data.

Investigate both Market-Data TCP connections.

Press Ctrl-C when you are finished.

```
[server] fd=5: peer closed (FIN, recv==0)
```

```
[server] closing fd=5 owner=2
```

```
[server] fd=6: peer closed (FIN, recv==0)
```

```
[server] closing fd=6 owner=3
```

```
[server] fd=7: peer closed (FIN, recv==0)
```

```
[server] closing fd=7 owner=4
```

```
[server] fd=8: peer closed (FIN, recv==0)
```

```
[server] closing fd=8 owner=5
```

Stopping Exchange Server...

```
root@exchange:/home/vijeta/COL-334-Assignment2- # grep -c '\[flush\]' /tmp/exp7.log # how many times it re
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # sysctl kern.ipc.maxsockbuf | tee /tmp/maxsockbuf_before.txt # record to restore later
```

```
kern.ipc.maxsockbuf: 2097152
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- #
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # sysctl kern.ipc.maxsockbuf=16384
```

```
kern.ipc.maxsockbuf: 2097152 -> 16384
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # sysctl net.inet.tcp.recvspace=4096
```

```
net.inet.tcp.recvspace: 4096 -> 4096
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # sysctl net.inet.tcp.sendspace=4096
```

```
net.inet.tcp.sendspace: 4096 -> 4096
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # sysctl net.inet.tcp.recvbuf_auto=0
```

```
net.inet.tcp.recvbuf_auto: 0 -> 0
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # sysctl net.inet.tcp.sendbuf_auto=0
```

```
net.inet.tcp.sendbuf_auto: 0 -> 0
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- #
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # sysctl kern.ipc.maxsockbuf net.inet.tcp.recvspace net.inet.tcp.sendspace \
```

```
> net.inet.tcp.recvbuf_auto net.inet.tcp.sendbuf_auto
```

```
kern.ipc.maxsockbuf: 16384
```

```
net.inet.tcp.recvspace: 4096
```

```
net.inet.tcp.sendspace: 4096
```

```
net.inet.tcp.recvbuf_auto: 0
```

```
net.inet.tcp.sendbuf_auto: 0
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # █
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # grep -c '\[flush\]' /tmp/exp7.log # how many times it refused
```

```
4519
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # grep '\[flush\]' /tmp/exp7.log | tail -3 # peak queued bytes
```

```
[flush] fd=6 send() EAGAIN, 76789 bytes still queued, enabling EVFILT_WRITE
```

```
[flush] fd=6 send() EAGAIN, 76806 bytes still queued, enabling EVFILT_WRITE
```

```
[flush] fd=6 send() EAGAIN, 76823 bytes still queued, enabling EVFILT_WRITE
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # grep -n 'win 0' /tmp/exp7.txt | grep -v 'Flags \[R\]' # zero-window advertisement
```

```
6334: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0x4511), seq 3122824128, ack 1424861366, win 0, options [no
```

```
p,nop,TS val 4291367894 ecr 3239482260], length 0
```

```
8160: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0x1dda), seq 3122824128, ack 1424861366, win 0, options [no
```

```
p,nop,TS val 4291372893 ecr 3239487300], length 0
```

```
10038: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0xf6c9), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291377893 ecr 3239492300], length 0
```

```
12010: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0xcfa7), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291382902 ecr 3239497309], length 0
```

```
13822: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0xa891), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291387905 ecr 3239502312], length 0
```

```
15672: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0x8181), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291392905 ecr 3239507312], length 0
```

```
18092: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0x4df7), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291399502 ecr 3239513909], length 0
```

```
22820: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0xe866), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291412502 ecr 3239526909], length 0
```

```
32228: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0x1ed4), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291438303 ecr 3239552710], length 0
```

```
51188: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0x8d34), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291489710 ecr 3239604117], length 0
```

```
72862: 127.0.0.1.31823 > 127.0.0.1.5000: Flags [.], cksum 0xfe28 (incorrect -> 0xb86c), seq 3122824128, ack 1424861366, win 0, options [n
```

```
op,nop,TS val 4291549713 ecr 3239664120], length 0
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # awk '$1=="tcp4" { if ($2+>mr[$4]) mr[$4]=$2+0; if ($3+>ms[$4]) ms[$4]=$3+0 }'
```

```
> END { for (k in mr) printf "%-26s maxRecvQ=%-8d maxSendQ=%d\n", k, mr[k], ms[k] }' \
```

```
> /tmp/exp7_states.log | sort
```

```
127.0.0.1.13333 maxRecvQ=38 maxSendQ=15
```

```
127.0.0.1.31823 maxRecvQ=4096 maxSendQ=0
```

```
127.0.0.1.48722 maxRecvQ=34 maxSendQ=0
```

```
127.0.0.1.49157 maxRecvQ=56 maxSendQ=32
```

```
127.0.0.1.5000 maxRecvQ=0 maxSendQ=4081
```

```
root@exchange:/home/vijeta/COL-334-Assignment2- # █
```

Observation

1. Both kernel buffers saturate at their limits. From `netstat -an -p tcp -x`, with the slow client on ephemeral port 31823:

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	R-HIWA	S-HIWA	R-BCNT	S-BCNT	rexmt	persist
tcp4	0	4081	127.0.0.1.5000	127.0.0.1.31823	4096	4096	0	4864	0.00	12.31
tcp4	4096	0	127.0.0.1.31823	127.0.0.1.5000	4096	4096	12800	0	0.00	0.00
tcp4	0	17	127.0.0.1.5000	127.0.0.1.48722	4096	4096	0	256	0.25	0.00
tcp4	17	0	127.0.0.1.48722	127.0.0.1.5000	4096	4096	256	0	0.00	0.00

Recv-Q 4096 against **R-HIWA 4096** on the slow client, and **Send-Q 4081** against **S-HIWA 4096** on the server side of that same connection — both at their ceilings. The normal client (port 48722) shows one 17-byte message in flight and nothing queued.

2. The persist timer is running on exactly one connection. **persist 12.31** appears on the server→slow-client row and reads **0.00** on every other row. This is the kernel's direct statement that this one connection, and only this one, is blocked by a closed receive window.

3. Zero-window advertisements with exponential backoff. Eleven pure-ACK segments were captured from the slow client, all carrying **Flags [.]** and **win 0**, and all with **identical seq 3122824128, ack 1424861366** — nothing is moving. The TCP timestamp values show the persist timer backing off exponentially and capping at 60 s:

Probe	TS val	Δ from previous
1	4291367894	—
2	4291372893	≈5.0 s
3	4291377893	≈5.0 s
4	4291382902	≈5.0 s
5	4291387905	≈5.0 s
6	4291392905	≈5.0 s
7	4291399502	≈6.6 s
8	4291412502	≈13.0 s
9	4291438303	≈25.8 s
10	4291489710	≈51.4 s
11	4291549713	≈60.0 s (capped)

4. The application-level refusal. **send()** returned **EAGAIN 4519 times**, on fd 6 only — never on the normal client's descriptor — with the userspace queue growing 17 bytes per refusal (one **TRADE** message):


```
[flush] fd=6 send() EAGAIN, 17 bytes still queued, enabling EVFILT_WRITE
[flush] fd=6 send() EAGAIN, 34 bytes still queued, enabling EVFILT_WRITE
...
[flush] fd=6 send() EAGAIN, 76823 bytes still queued, enabling EVFILT_WRITE
```

5. The bytes account for themselves exactly.

76,823	bytes in the server's userspace OutBuffer	(peak [flush] line)
4,081	bytes in the server's kernel send buffer	(Send-Q, limit 4096)
4,096	bytes in the slow client's kernel receive buffer	(Recv-Q, at R-HIWA)
85,000	= 5000 trades × 17 bytes	

Every byte the exchange generated is accounted for in one of the three places it can be when a receiver stops reading. Note also **R-BCNT 12800** against **Recv-Q 4096**: the kernel charges socket buffer usage in mbuf clusters, so the memory *charge* exceeds the payload byte count.

6. No other client was affected. The normal subscriber held **Recv-Q 17**, **Send-Q 0**, **persist 0.00** throughout and received all 5000 updates; both traders continued transacting to completion (**13333** and **49157** show live, draining queues); the harness reported **Generated 5000 matching trades**.

Answer

As the slow client stops reading, its receive buffer fills to its high-water mark and it advertises a **zero window**. TCP flow control then forbids the server's stack from sending further data, so the server's send buffer fills and the connection enters the **persist state**, emitting window probes with exponential backoff ($\approx 5\text{ s} \rightarrow 6.6 \rightarrow 13 \rightarrow 25.8 \rightarrow 51.4 \rightarrow 60\text{ s}$, capped), directly visible as **persist 12.31** in **netstat -x**. At that point **send()** on the non-blocking descriptor returns **EAGAIN** — 4519 times over the run — and the server retains the undelivered data in that connection's own **OutBuffer**, peaking at 76,823 bytes, enables **EVFILT_WRITE** for that descriptor alone, and returns to the event loop.

The three quantities sum to exactly 85,000 bytes: the complete trade stream, distributed across the server's userspace buffer, the server's kernel send buffer, and the client's kernel receive buffer.

This does not affect the server's ability to communicate with other clients. Throughout, the normal subscriber's connection showed **Send-Q 0** and **persist 0.00**, received every update, and both traders completed all 5000 trades. Backpressure was confined entirely to the offending connection. Had the socket been blocking, that same **send()** would have slept inside the kernel and the single-threaded server would have stalled for all four clients until the slow reader drained — which it never does. The residual risk is therefore memory rather than availability, which is why the per-connection output buffer is capped at 8 MB and a client exceeding it is disconnected; that cap was not approached here, as 76 KB $\ll$ 8 MB.

8. Experiment 8 — Unexpected Client Disconnection

Question. When a client disappears unexpectedly while the Exchange Server is communicating with it, what happens to their TCP connection, and what network-level evidence allows you to determine how the server detects the failure?

Approach

A separate helper process subscribes to JNST and then sleeps without reading, while a second subscriber drains normally and two traders generate activity. After 20 trades the helper is killed with **SIGKILL**, giving it no opportunity to run any shutdown code, and 50 further trades are generated at 50 ms intervals.

The investigation was built around a before/after comparison of the server's own descriptor table, so that the effect of the kill could be isolated from the ordinary end-of-run teardown, combined with a packet trace filtered to termination segments only.

Commands and tools

```
# Baseline: the keepalive timers that would govern detection WITHOUT active
traffic
sysctl net.inet.tcp.always_keepalive net.inet.tcp.keepidle
net.inet.tcp.keepintvl

# T4 – attach the syscall trace as the server spawns
while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done
ktrace -i -f /tmp/exp8.kt -p $PID -t c

# T2 – both pollers
~/tools/poll-tcp.sh 0.2 /tmp/exp8.states & ~/tools/poll-tcp.sh 0.5

# T3 – packet capture
tcpdump -l -i lo0 -n -S -vv -tttt 'tcp port 5000' > /tmp/exp8_packets.txt

# T1 – harness
SX_VERBOSE=1 python3 -u experiment.py 8 2>&1 | tee /tmp/exp8.log

# T5 – BEFORE the kill: identify the helper and record the server's
descriptor table
PID=$(pgrep -n exchange_server)
sockstat -4 -p 5000
procstat -f $PID | grep TCP | tee /tmp/exp8.fmap.before
netstat -an -p tcp | egrep 'Proto|\.5000'

# T5 – AFTER T1 prints "The client process has disappeared."
PID=$(pgrep -n exchange_server)
netstat -an -p tcp | egrep 'Proto|\.5000'
procstat -f $PID | grep TCP | tee /tmp/exp8.fmap.after
diff /tmp/exp8.fmap.before /tmp/exp8.fmap.after
```

```
# T4 – post-run extraction
ktrace -C
kdump -f /tmp/exp8.kt -T | egrep 'kevent|recvfrom|sendto|close' | tail -40
grep -E 'reset by peer|peer closed|closing fd' /tmp/exp8.log
cat /tmp/exp8_packets.txt | egrep 'Flags \[R|Flags \[F'
```

Screenshots

```
root@exchange:/home/cooluser1 # PID=$(pgrep -n exchange_server); sockstat -4 -p 5000; procstat -f $PID | grep TCP | tee /tmp/exp8.fmap
before; netstat -an -p tcp | egrep 'Proto|.5000'
USER      COMMAND  PID    FD    PROTO  LOCAL ADDRESS    FOREIGN ADDRESS
root      python3.12 32017  3     tcp4   127.0.0.1:43707   127.0.0.1:5000
root      exchange_s 32013  3     tcp4   127.0.0.1:5000    *:
root      exchange_s 32013  5     tcp4   127.0.0.1:5000    127.0.0.1:19136
root      exchange_s 32013  6     tcp4   127.0.0.1:5000    127.0.0.1:42619
root      exchange_s 32013  7     tcp4   127.0.0.1:5000    127.0.0.1:21668
root      exchange_s 32013  8     tcp4   127.0.0.1:5000    127.0.0.1:43707
root      python3.12 32005  3     tcp4   127.0.0.1:19136   127.0.0.1:5000
root      python3.12 32005  4     tcp4   127.0.0.1:42619   127.0.0.1:5000
root      python3.12 32005  5     tcp4   127.0.0.1:21668   127.0.0.1:5000
32013 exchange_server  3 s - rw---n--  2      0 TCP 0 0 127.0.0.1:5000 *:0
32013 exchange_server  5 s - rw---n--  3      0 TCP 17 0 127.0.0.1:5000 127.0.0.1:19136
32013 exchange_server  6 s - rw---n--  3      0 TCP 36 0 127.0.0.1:5000 127.0.0.1:42619
32013 exchange_server  7 s - rw---n--  3      0 TCP 34 0 127.0.0.1:5000 127.0.0.1:21668
32013 exchange_server  8 s - rw---n--  3      0 TCP 17 0 127.0.0.1:5000 127.0.0.1:43707
Proto      Recv-Q Send-Q Local Address    Foreign Address  (state)
tcp4       0      17 127.0.0.1.5000   127.0.0.1.43707 ESTABLISHED
tcp4       340    0 127.0.0.1.43707 127.0.0.1.5000   ESTABLISHED
tcp4       0      34 127.0.0.1.5000   127.0.0.1.21668 ESTABLISHED
tcp4       0      0 127.0.0.1.21668 127.0.0.1.5000   ESTABLISHED
tcp4       0      36 127.0.0.1.5000   127.0.0.1.42619 ESTABLISHED
tcp4       0      0 127.0.0.1.42619 127.0.0.1.5000   ESTABLISHED
tcp4       0      17 127.0.0.1.5000   127.0.0.1.19136 ESTABLISHED
tcp4       0      0 127.0.0.1.19136 127.0.0.1.5000   ESTABLISHED
tcp4       0      0 127.0.0.1.5000   *:
tcp4       0      0 127.0.0.1.5000   *:
root@exchange:/home/cooluser1 # PID=$(pgrep -n exchange_server); netstat -an -p tcp | egrep 'Proto|.5000'; procstat -f $PID | grep TCP
| tee /tmp/exp8.fmap.after; diff /tmp/exp8.fmap.before /tmp/exp8.fmap.after
Proto      Recv-Q Send-Q Local Address    Foreign Address  (state)
tcp4       0      16 127.0.0.1.5000   127.0.0.1.21668 ESTABLISHED
tcp4       16     0 127.0.0.1.21668 127.0.0.1.5000   ESTABLISHED
tcp4       0      0 127.0.0.1.5000   127.0.0.1.42619 ESTABLISHED
tcp4       18     0 127.0.0.1.42619 127.0.0.1.5000   ESTABLISHED
tcp4       0      0 127.0.0.1.5000   127.0.0.1.19136 ESTABLISHED
tcp4       0      0 127.0.0.1.19136 127.0.0.1.5000   ESTABLISHED
tcp4       0      0 127.0.0.1.5000   *:
tcp4       0      0 127.0.0.1.5000   *:
32013 exchange_server  3 s - rw---n--  2      0 TCP 0 0 127.0.0.1:5000 *:0
32013 exchange_server  5 s - rw---n--  3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:19136
32013 exchange_server  6 s - rw---n--  3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:42619
32013 exchange_server  7 s - rw---n--  3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:21668
2,5c2,4
< 32013 exchange_server  5 s - rw---n--  3      0 TCP 17 0 127.0.0.1:5000 127.0.0.1:19136
< 32013 exchange_server  6 s - rw---n--  3      0 TCP 36 0 127.0.0.1:5000 127.0.0.1:42619
< 32013 exchange_server  7 s - rw---n--  3      0 TCP 34 0 127.0.0.1:5000 127.0.0.1:21668
< 32013 exchange_server  8 s - rw---n--  3      0 TCP 17 0 127.0.0.1:5000 127.0.0.1:43707
---
> 32013 exchange_server  5 s - rw---n--  3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:19136
> 32013 exchange_server  6 s - rw---n--  3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:42619
> 32013 exchange_server  7 s - rw---n--  3      0 TCP 0 0 127.0.0.1:5000 127.0.0.1:21668
root@exchange:/home/cooluser1 # cat /tmp/exp8_packets.txt | egrep 'Flags \[R|Flags \[F'
127.0.0.1.5000 > 127.0.0.1.64164: Flags [R.], cksum 0xfe1c (incorrect -> 0x55d5), seq 0, ack 1486026041, win 0, length 0
127.0.0.1.46431 > 127.0.0.1.5000: Flags [R.], cksum 0xfe28 (incorrect -> 0x1dcb), seq 1145275301, ack 1972288388, win 0, options [nop,nop,TS val 2272893218 ecr 2680608933], length 0
127.0.0.1.43707 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0x3980), seq 1875927223, ack 3462142896, win 1277, options [nop,nop,TS val 1924869341 ecr 3942791785], length 0
127.0.0.1.5000 > 127.0.0.1.43707: Flags [F.], cksum 0xfe28 (incorrect -> 0x3870), seq 3462142896, ack 1875927224, win 1277, options [nop,nop,TS val 3942791928 ecr 1924869341], length 0
127.0.0.1.19136 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0x3565), seq 3651131453, ack 1196199771, win 1277, options [nop,nop,TS val 358456932 ecr 358456932], length 0
127.0.0.1.42619 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0x77f9), seq 3738141868, ack 143943145, win 1277, options [nop,nop,TS val 1580295235 ecr 1623674083], length 0
127.0.0.1.5000 > 127.0.0.1.19136: Flags [F.], cksum 0xfe28 (incorrect -> 0xfa34), seq 1196199771, ack 3651131454, win 1277, options [nop,nop,TS val 3383217684 ecr 358456932], length 0
127.0.0.1.21668 > 127.0.0.1.5000: Flags [F.], cksum 0xfe28 (incorrect -> 0x17ce), seq 661590372, ack 1967437538, win 1277, options [nop,nop,TS val 1156384527 ecr 2567940540], length 0
127.0.0.1.5000 > 127.0.0.1.42619: Flags [F.], cksum 0xfe28 (incorrect -> 0x3ce3), seq 143943145, ack 3738141869, win 1277, options [nop,nop,TS val 1623689119 ecr 1580295235], length 0
127.0.0.1.5000 > 127.0.0.1.21668: Flags [F.], cksum 0xfe28 (incorrect -> 0xdcc0), seq 1967437538, ack 661590373, win 1277, options [nop,nop,TS val 2567955656 ecr 1156384527], length 0
root@exchange:/home/cooluser1 #
```

```
[root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # SX_VERBOSE=1 python3 -u experiment.py 8 2>&1 | tee /tmp/exp8.log
=== Experiment 8: Unexpected Client Disconnection ===

Started Exchange Server (PID 32013).
[server] listening on 127.0.0.1:5000 (fd=3, kq=4)
Exchange Server is listening.
[server] accepted fd=5 owner=1 from 127.0.0.1:46431

[server] fd=5: connection reset by peer (RST, ECONNRESET)
[server] closing fd=5 owner=1
Surviving Market-Data Client: connected from 127.0.0.1:19136
[server] accepted fd=5 owner=2 from 127.0.0.1:19136
[recv] fd=5 n=15 data=SUBSCRIBE JNST

[server] accepted fd=6 owner=3 from 127.0.0.1:42619
Buyer Trader: connected from 127.0.0.1:42619
Seller Trader: connected from 127.0.0.1:21668
[server] accepted fd=7 owner=4 from 127.0.0.1:21668
[recv] fd=6 n=23 data=LOGIN experiment_buyer

[recv] fd=7 n=24 data=LOGIN experiment_seller

[server] accepted fd=8 owner=5 from 127.0.0.1:43707
[recv] fd=8 n=15 data=SUBSCRIBE JNST

Both Market-Data Clients should now be connected.
Generating traffic before the disconnection...

[recv] fd=6 n=15 data=BUY JNST 1 238

[recv] fd=7 n=16 data=SELL JNST 1 238
```

```
Experiment finished.
[server] fd=5: peer closed (FIN, recv==0)
[server] closing fd=5 owner=2
[server] fd=6: peer closed (FIN, recv==0)
[server] closing fd=6 owner=3
[server] fd=7: peer closed (FIN, recv==0)
Stopping Exchange Server...[server] closing fd=7 owner=4

[root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # pkill -f poll-tcp
```



```

root@exchange:/home/cooluser1 # while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done; ktrace -i -f /tmp/exp8.kt -p $PID -t c
root@exchange:/home/cooluser1 # ktrace -C
root@exchange:/home/cooluser1 # kdump -f /tmp/exp8.kt -T | egrep 'kevent|recvfrom|sendto|close' | tail -40
32013 exchange_server 1789032214.007256664 CALL recvfrom(0x7,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032214.007258289 RET recvfrom -1 errno 35 Resource temporarily unavailable
32013 exchange_server 1789032214.007260581 CALL kevent(0x4,0,0,0x810af390,0x40,0)
32013 exchange_server 1789032214.100087414 RET kevent 2
32013 exchange_server 1789032214.100104331 CALL recvfrom(0x6,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032214.100107998 RET recvfrom 15/0xf
32013 exchange_server 1789032214.100167206 CALL sendto(0x6,0x6ab8f7809101,0x13,0,0,0)
32013 exchange_server 1789032214.100181123 RET sendto 19/0x13
32013 exchange_server 1789032214.100183706 CALL recvfrom(0x6,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032214.100185123 RET recvfrom -1 errno 35 Resource temporarily unavailable
32013 exchange_server 1789032214.100186248 CALL recvfrom(0x7,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032214.100187873 RET recvfrom 16/0x10
32013 exchange_server 1789032214.100195373 CALL sendto(0x7,0x6ab8f78091a1,0x13,0,0,0)
32013 exchange_server 1789032214.100202539 RET sendto 19/0x13
32013 exchange_server 1789032214.100205123 CALL sendto(0x6,0x6ab8f7809101,0x12,0,0,0)
32013 exchange_server 1789032214.100206456 RET sendto 18/0x12
32013 exchange_server 1789032214.100207789 CALL sendto(0x7,0x6ab8f78091a1,0x10,0,0,0)
32013 exchange_server 1789032214.100233748 RET sendto 16/0x10
32013 exchange_server 1789032214.100235206 CALL sendto(0x5,0x6ab8f7809061,0x11,0,0,0)
32013 exchange_server 1789032214.100242164 RET sendto 17/0x11
32013 exchange_server 1789032214.100243498 CALL recvfrom(0x7,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032214.100244498 RET recvfrom -1 errno 35 Resource temporarily unavailable
32013 exchange_server 1789032214.100245789 CALL kevent(0x4,0,0,0x810af390,0x40,0)
32013 exchange_server 1789032229.252425206 RET kevent 1
32013 exchange_server 1789032229.252586039 CALL recvfrom(0x5,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032229.252598581 RET recvfrom 0
32013 exchange_server 1789032229.252733414 CALL close(0x5)
32013 exchange_server 1789032229.252747831 RET close 0
32013 exchange_server 1789032229.252828914 CALL kevent(0x4,0,0,0x810af390,0x40,0)
32013 exchange_server 1789032229.252838123 RET kevent 2
32013 exchange_server 1789032229.252840581 CALL recvfrom(0x6,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032229.252843081 RET recvfrom 0
32013 exchange_server 1789032229.252911789 CALL recvfrom(0x7,0x810ae308,0x1000,0,0,0)
32013 exchange_server 1789032229.252913623 RET recvfrom 0
32013 exchange_server 1789032229.252980706 CALL close(0x6)
32013 exchange_server 1789032229.252986248 RET close 0
32013 exchange_server 1789032229.252988081 CALL close(0x7)
32013 exchange_server 1789032229.252991206 RET close 0
32013 exchange_server 1789032229.252992998 CALL kevent(0x4,0,0,0x810af390,0x40,0)
32013 exchange_server 1789032229.253203123 RET kevent -1 errno 4 Interrupted system call
root@exchange:/home/cooluser1 # grep -E 'reset by peer|peer closed|closing fd' /tmp/exp8.log
[server] fd=5: connection reset by peer (RST, ECONNRESET)
[server] closing fd=5 owner=1
[server] fd=8: peer closed (FIN, recv==0)
[server] closing fd=8 owner=5
[server] fd=5: peer closed (FIN, recv==0)
[server] closing fd=5 owner=2
[server] fd=6: peer closed (FIN, recv==0)
[server] closing fd=6 owner=3
[server] fd=7: peer closed (FIN, recv==0)

```

```

root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # sysctl net.inet.tcp.always_keepalive net.inet.tcp.keepidle net.inet.tcp.keepintvl
net.inet.tcp.always_keepalive: 1
net.inet.tcp.keepidle: 720000
net.inet.tcp.keepintvl: 75000
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #

```

Observation

1. Identifying the victim. `sockstat` shows two distinct Python processes: PID 32017 holding a single connection from ephemeral port **43707**, and PID 32005 holding three connections from ports 19136, 42619 and 21668. The single-connection process is the disappearing helper; the server's descriptor for it is **fd 8**.

2. The helper was demonstrably not reading. In the "before" snapshot, its connection is the only one with a non-empty receive queue:

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	(state)	
tcp4	0	17	127.0.0.1.5000	127.0.0.1.43707	ESTABLISHED	
tcp4	340	0	127.0.0.1.43707	127.0.0.1.5000	ESTABLISHED	<-
340 bytes unread						
tcp4	0	0	127.0.0.1.21668	127.0.0.1.5000	ESTABLISHED	
tcp4	0	0	127.0.0.1.42619	127.0.0.1.5000	ESTABLISHED	
tcp4	0	0	127.0.0.1.19136	127.0.0.1.5000	ESTABLISHED	

340 bytes is exactly 20×17 , the twenty **TRADE INST 1 238** messages delivered before the kill and never consumed.

3. Exactly one descriptor disappears. The **diff** of the before/after **procstat -f** output is unambiguous — fd 8 (port 43707) is gone, and fds 5, 6 and 7 survive with their queues drained to 0:

```
< 32013 exchange_server  5 s - rw---n--  3  0 TCP 17 0 127.0.0.1:5000
127.0.0.1:19136
< 32013 exchange_server  6 s - rw---n--  3  0 TCP 36 0 127.0.0.1:5000
127.0.0.1:42619
< 32013 exchange_server  7 s - rw---n--  3  0 TCP 34 0 127.0.0.1:5000
127.0.0.1:21668
< 32013 exchange_server  8 s - rw---n--  3  0 TCP 17 0 127.0.0.1:5000
127.0.0.1:43707
---
> 32013 exchange_server  5 s - rw---n--  3  0 TCP  0 0 127.0.0.1:5000
127.0.0.1:19136
> 32013 exchange_server  6 s - rw---n--  3  0 TCP  0 0 127.0.0.1:5000
127.0.0.1:42619
> 32013 exchange_server  7 s - rw---n--  3  0 TCP  0 0 127.0.0.1:5000
127.0.0.1:21668
```

The "after" **netstat** likewise shows the 43707 4-tuple absent from both sides while all three surviving connections remain ESTABLISHED.

4. The termination segment was a FIN, not an RST. Filtering the capture to FIN and RST segments gives, in chronological order:

```
127.0.0.1.5000 > 127.0.0.1.64164: Flags [R.], seq 0, ack 1486026041, win
0, length 0
127.0.0.1.46431 > 127.0.0.1.5000 : Flags [R.], seq 1145275301, ack
1972288388, win 0, length 0
127.0.0.1.43707 > 127.0.0.1.5000 : Flags [F.], seq 1875927223, ack
3462142096, win 1277    <- the kill
127.0.0.1.5000 > 127.0.0.1.43707: Flags [F.], seq 3462142096, ack
1875927224, win 1277
127.0.0.1.19136 > 127.0.0.1.5000 : Flags [F.], ...
\
127.0.0.1.42619 > 127.0.0.1.5000 : Flags [F.], ...
| end-of-run
127.0.0.1.5000 > 127.0.0.1.19136: Flags [F.], ...
| teardown
127.0.0.1.21668 > 127.0.0.1.5000 : Flags [F.], ...
|
127.0.0.1.5000 > 127.0.0.1.42619: Flags [F.], ...
|
127.0.0.1.5000 > 127.0.0.1.21668: Flags [F.], ...
/
```

The two RSTs are unrelated to the kill: they belong to the harness readiness probe described in §0.5, on ports 64164 and 46431, neither of which appears in the descriptor table. The killed client's connection on port 43707 terminated with an **orderly FIN exchange**, and it appears in the trace before the other three FIN pairs, which is consistent with the kill occurring mid-run while those three closed only at the end.

5. Server-side detection and continuity. The server logged the event on fd 8 as a peer close with `recv()` returning 0, called `close_conn()`, and continued. The 50 post-kill trades completed normally, and the surviving subscriber received every one of them.

Answer

When the client process is killed, the kernel closes all of its open descriptors on its behalf. On this system that close performed an **orderly disconnect**: the packet trace shows a FIN from the dead client's port 43707, the server's ACK-plus-FIN in reply, and no RST anywhere near the kill. The two RSTs in the capture belong to the harness's startup probe and are excluded.

This is worth stating precisely because it contradicts a plausible prediction. The helper had 340 bytes sitting unread in its receive buffer at the moment it died, and some TCP stacks answer a close with pending unread data by emitting an RST rather than a FIN. **That behaviour was not observed here:** FreeBSD 14.4 performed a normal disconnect and sent FIN despite the unread data. The experiment therefore demonstrates that a process dying abruptly does not by itself produce an abrupt *connection* termination — the signal that kills the process and the segment the stack emits are independent, and the latter is decided by the kernel's close path.

The server detects the failure through its ordinary event path: `kqueue` reports the descriptor readable with `EV_EOF` set, and `recv()` returns 0, which is indistinguishable at the socket layer from a well-behaved client calling `close()`. The server routes this into `close_conn()`, releasing the descriptor, the username and the `owner_id` → `fd` mapping while leaving any resting orders in the book per §2.6. Had the stack emitted an RST instead, the same event would have surfaced as `recv()` returning -1 with `ECONNRESET`, and had the server been mid-write, as `EPIPE`; all three paths converge on the same handler, and because `SIGPIPE` is ignored none of them can terminate the server. This is why the implementation treats "peer gone" as a single condition with three possible signatures rather than assuming any one of them.

Finally, detection was immediate **only because the server was actively writing to that socket**. A peer that vanished silently on an otherwise idle connection would not be detected until TCP keepalives fired — `net.inet.tcp.keepidle` defaults to 7200000 ms, i.e. two hours. Prompt detection of a dead peer is a property of ongoing traffic, not of TCP alone.

9. Section 6.9 — Bonus: Connection Scalability and I/O Design

9.1 The architectural constraint that governs this task

Before any resource measurement, two structural facts determine whether 70,000 connections is reachable at all:

Both endpoints are local. Server and generator both run inside the VM, so N connections consume $2N$ TCP protocol control blocks, $2N$ `struct sockets` and $\sim 2N$ system-wide file descriptors. Every limit had to be sized against 140,000, not 70,000. It is also why each connection appears twice in `netstat`, and why the counts below filter on the server's local port.

A single 4-tuple space cannot hold 70,000 connections. With `127.0.0.1:5000` fixed as the destination and a single source address, the only free field is the client's ephemeral port. The FreeBSD default range (`net.inet.ip.portrange.first=10000 ... last=65535`) yields $\sim 55,535$, and even the widest usable range yields only $\sim 64,500$. This ceiling is architectural, not a resource limit.

9.2 Configuration

```
# /boot/loader.conf (kern.ipc.maxsockets is RDTUN and cannot be set at
runtime)
kern.maxfiles="400000"
kern.ipc.maxsockets="300000"
kern.ipc.nmbclusters="262144"

# runtime
sysctl kern.maxfilesperproc=200000
sysctl kern.ipc.somaxconn=4096
sysctl net.inet.ip.portrange.first=1024
sysctl net.inet.ip.portrange.randomized=0      # sequential allocation;
randomized search                             # degrades sharply once the
range is nearly full
sysctl net.inet.tcp.sendspace=4096 net.inet.tcp.recvspace=4096
sysctl net.inet.tcp.sendbuf_auto=0 net.inet.tcp.recvbuf_auto=0
ulimit -n 200000                             # in both the server and
generator shells
```

The defaults these were changed from are the ones recorded in §0.4.

9.3 The client-generation program

`tools/connflood.cpp` creates and maintains the required number of simultaneous idle connections and exchanges no application-level data. Its design points:

- raises its own `RLIMIT_NOFILE` to the hard maximum via `setrlimit` and reports the result, since the generator needs as many descriptors as the server;
- issues non-blocking `connect()` in batches, collecting completions through its own `kqueue` so that the accept queue is never overwhelmed;
- accepts `--src a,b,c` to round-robin the *source* address across loopback aliases, which is how the 4-tuple ceiling of §9.1 is overcome;
- classifies and reports the first hard failure by `errno` — `EMFILE`, `ENFILE`, `EADDRNOTAVAIL`, `EADDRINUSE`, `ECONNREFUSED` — together with the exact count reached, which is what makes question 1 answerable precisely;
- parks and holds all descriptors open after reaching the target, so measurements are taken while the connections are genuinely established and idle.


```
./connflood 127.0.0.1 5000 70000 --src 127.0.0.1,127.0.0.2,127.0.0.3 --
report 5000
```

9.3.1 The measurement procedure

Three terminals were used, and the full four-step sequence was repeated once per rung for 10,000 / 20,000 / 30,000 / 40,000 / 50,000 / 60,000 / 70,000.

Step 1 — restart the server (T1). The server is restarted before every rung so that RSS reflects only the connections of that rung and is not cumulative. Output is discarded so the terminal stays readable:

```
pkill exchange_server
ulimit -n 200000
./server/run-server 127.0.0.1 5000 > /dev/null 2>&1
```

Step 2 — launch the generator (T2). `--src` distributes the ephemeral ports across the three loopback addresses, which is what bypasses the ceiling established in §9.5.1. `--batch 64` paces the connect rate so the accept queue is not overrun:

```
ulimit -n 200000
./connflood 127.0.0.1 5000 10000 --src 127.0.0.1,127.0.0.2,127.0.0.3 \
--report 2500 --batch 64 2>&1 | tee /tmp/gen_10000.log
```

Wait for `holding connections open and idle`. Take measurements now before proceeding — this is what guarantees the measurement is taken while all connections are simultaneously established and idle, as the handout requires.

Step 3 — measure (T3). Taken while the generator holds the connections open:

```
src/tools/measure.sh 10000 | tee /tmp/measure_10000.txt
```

`measure.sh` captures the connection count, server RSS/VSZ/CPU, the server's and the generator's descriptor counts, `kern.openfiles` against `kern.maxfiles`, `netstat -m` mbuf usage, and the `vmstat -z` socket and PCB zones, then appends one CSV row to `/tmp/bonus.csv`. The rows accumulate across rungs and become the table in §9.4 directly.

Connections are counted with

```
netstat -an -p tcp | awk '$4 ~ /\.5000$/ && $6=="ESTABLISHED"' | wc -l
```

filtering on the server's local port, because on loopback every connection otherwise appears twice (§9.1).

Step 4 — teardown (T2). Ctrl-C releases the generator's descriptors; the same count command is then re-run to confirm the connections drained cleanly to 0 before the next rung begins.

9.4 Resource measurement table

Idle connections	Server RSS	Server CPU	Server open fds	System-wide open fds	Socket / PCB zone usage	Max established
10,000	5,304 KB	0.0 %	10,010	30,111	40,040 / 300,000	10,000
20,000	6,964 KB	0.0 %	20,010	50,111	60,040 / 300,000	20,000
30,000	8,728 KB	0.5 %	30,010	70,111	80,040 / 300,000	30,000
40,000	10,300 KB	0.6 %	40,010	90,111	100,040 / 300,000	40,000
50,000	11,888 KB	0.4 %	50,010	110,111	120,040 / 300,000	50,000
60,000	13,868 KB	0.2 %	60,010	130,111	140,040 / 300,000	60,000
70,000	15,448 KB	0.2 %	70,010	150,111	160,040 / 300,000	70,000

Two observations from the table itself:

- **Server open fds = N + 10 at every level**, confirming one descriptor per connection plus the listener, the kqueue, stdio and a small constant. **System-wide open fds $\approx 2N + 10,111$** , which is the two-endpoints-per-connection effect of §9.1 made visible.
- **mbuf cluster usage remained at 0 in use throughout**. Idle connections carry no data, so no packet buffers are allocated; the entire per-connection cost is in socket and protocol-control-block structures. Socket-buffer *limits* (`sb_hiwat`) are accounting ceilings, not allocations.

Screenshots

```
root@exchange:/home/cooluser1 # cd A2_2024CS10573_2024CS10836/
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # src/tools/measure.sh 10000 | tee /tmp/measure_10000.txt
=====
BONUS MEASUREMENT label=10000 2026-09-10 18:01:02
server pid=1577 generator pid=1593
=====
-- connections --
established to server port 5000 : 10001
net.inet.tcp.pcbcount (all PCBs): 20007 # ~2x EST on loopback
-- server memory / cpu --
pid=,rss=,vsz=,%cpu=,%mem=,time=,nlwp= rss= KB vsz= KB cpu=% mem=% time= threads=
pid=1577 rss= KB vsz= KB cpu=% mem=% time= threads=
-- kernel UMA zones (ITEM / USED / SIZE) --
ITEM SIZE LIMIT USED FREE REQ FAIL SLEEP XDOM
mbuf_packet: 256, 0, 0, 762, 7853, 0, 0, 0
mbuf: 256,1624107, 577, 1211, 1216533, 0, 0, 0
mbuf_cluster: 2048, 262144, 762, 0, 762, 0, 0, 0
mbuf_jumbo_page: 4096, 126883, 320, 696, 83651, 0, 0, 0
mbuf_jumbo_9k: 9216, 37595, 0, 0, 0, 0, 0, 0
mbuf_jumbo_16k: 16384, 21147, 0, 0, 0, 0, 0, 0
socket: 1152, 300000, 20030, 35494, 274851, 0, 0, 0
tcp_inpcb: 1320, 0, 20007, 37443, 272468, 0, 0, 0
tcp_inpcb ports: 32, 0, 10001, 26791, 136265, 0, 0, 0
tcp_log: 408,500000, 0, 0, 0, 0, 0, 0
tcp_log_id_bucket: 176, 0, 0, 0, 0, 0, 0, 0
tcp_log_id_node: 120, 0, 0, 0, 0, 0, 0, 0
(kernel bytes for a zone = USED x SIZE; subtract your idle baseline)
-- file descriptors --
server TCP descriptors : 10001
server total descriptors : 10009
generator TCP descriptors : 10000
system-wide kern.openfiles : 20115 / kern.maxfiles=400000
kern.maxfilesperproc : 200000
shell RLIMIT_NOFILE (ulimit -n) : 200000
-- socket buffers / mbufs --
577/1973/2550 mbufs in use (current/cache/total)
0/762/762/262144 mbuf clusters in use (current/cache/total/max)
320/696/1016/126883 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37595 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21147 16k jumbo clusters in use (current/cache/total/max)
1424K/4801K/6225K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sfbufs denied
0 requests for sfbufs delayed
kern.ipc.maxsockbuf : 2097152
kern.ipc.maxsockets : 300000
net.inet.tcp.sendspace/recvspace: 32768/65536
-- system memory --
0 0 0 635M 3.5G 8 0 0 0 0 10 1 0 186 344 366 0 0 98
4263751680 906113
CSV row appended to /tmp/bonus.csv:
0/0/0/21147,20007
```

```
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # ulimit -n 200000
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # src/tools/measure.sh 20000 | tee /tmp/measure_20000.txt
=====
BONUS MEASUREMENT   label=20000   2026-09-10 18:03:55
server pid=1663     generator pid=1665
=====
-- connections --
established to server port 5000 : 20000
net.inet.tcp.pcbcount (all PCBs): 40007   # ~2x EST on loopback
-- server memory / cpu --
pid=,rss=,vsz=,%cpu=,%mem=,time=,nlwp= rss= KB vsz= KB cpu=% mem=% time= threads=
pid=1663 rss= KB vsz= KB cpu=% mem=% time= threads=
-- kernel UMA zones (ITEM / USED / SIZE) --
ITEM                SIZE  LIMIT   USED    FREE    REQ FAIL SLEEP XDOM
mbuf_packet:         256,    0,      0,      762,    7917,    0,    0,    0
mbuf:                256,1624107,  577,    1211, 1324201,  0,    0,    0
mbuf_cluster:        2048, 262144,  762,      0,    762,    0,    0,    0
mbuf_jumbo_page:     4096, 126883,  320,    696,   86281,  0,    0,    0
mbuf_jumbo_9k:       9216,  37595,    0,      0,      0,    0,    0,    0
mbuf_jumbo_16k:      16384, 21147,    0,      0,      0,    0,    0,    0
socket:              1152, 300000, 40030, 15417, 314858,  0,    0,    0
tcp_inpcb:           1320,    0,   40007, 16168, 312469,  0,    0,    0
tcp_inpcb ports:      32,    0,   20002, 16790, 156266,  0,    0,    0
tcp_log:             408,5000000,    0,      0,      0,    0,    0,    0
tcp_log_id_bucket:    176,    0,      0,      0,      0,    0,    0,    0
tcp_log_id_node:      120,    0,      0,      0,      0,    0,    0,    0
      (kernel bytes for a zone = USED x SIZE; subtract your idle baseline)
-- file descriptors --
server TCP descriptors      : 20001
server total descriptors   : 20009
generator TCP descriptors   : 20000
system-wide kern.openfiles  : 40115 / kern.maxfiles=400000
kern.maxfilesperproc        : 200000
shell RLIMIT_NOFILE (ulimit -n) : 200000
-- socket buffers / mbufs --
577/1973/2550 mbufs in use (current/cache/total)
0/762/762/262144 mbuf clusters in use (current/cache/total/max)
320/696/1016/126883 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37595 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21147 16k jumbo clusters in use (current/cache/total/max)
1424K/4801K/6225K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sbufs denied
0 requests for sbufs delayed
kern.ipc.maxsockbuf        : 2097152
kern.ipc.maxsockets        : 300000
net.inet.tcp.sendspace/recvspace: 32768/65536
-- system memory --
0 0 0 635M 3.5G 8 0 0 0 0 10 0 0 174 384 326 0 0 99
4263751680 905525
CSV row appended to /tmp/bonus.csv:
0/0/0/21147,40007
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #
```



```
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # src/tools/measure.sh 30000 | tee /tmp/measure_30000.txt
=====
BONUS MEASUREMENT label=30000 2026-09-10 18:06:22
server pid=1726 generator pid=1730
=====
-- connections --
established to server port 5000 : 30000
net.inet.tcp.pcbcount (all PCBs): 60007 # ~2x EST on loopback
-- server memory / cpu --
pid=rss=,vsz=,%cpu=,%mem=,time=,nlwp= rss= KB vsz= KB cpu=% mem=% time= threads=
pid=1726 rss= KB vsz= KB cpu=% mem=% time= threads=
-- kernel UMA zones (ITEM / USED / SIZE) --
ITEM SIZE LIMIT USED FREE REQ FAIL SLEEP XDOM
mbuf_packet: 256, 0, 0, 762, 7977, 0, 0, 0
mbuf: 256,1624107, 577, 1211, 1505966, 0, 0, 0
mbuf_cluster: 2048, 262144, 762, 0, 762, 0, 0, 0
mbuf_jumbo_page: 4096, 126883, 320, 696, 91871, 0, 0, 0
mbuf_jumbo_9k: 9216, 37595, 0, 0, 0, 0, 0, 0
mbuf_jumbo_16k: 16384, 21147, 0, 0, 0, 0, 0, 0
socket: 1152, 300000, 60030, 170, 374866, 0, 0, 0
tcp_inpcb: 1320, 0, 60007, 44, 372470, 0, 0, 0
tcp_inpcb ports: 32, 0, 30002, 6790, 186267, 0, 0, 0
tcp_log: 408,500000, 0, 0, 0, 0, 0, 0
tcp_log_id_bucket: 176, 0, 0, 0, 0, 0, 0, 0
tcp_log_id_node: 120, 0, 0, 0, 0, 0, 0, 0
(kernel bytes for a zone = USED x SIZE; subtract your idle baseline)
-- file descriptors --
server TCP descriptors : 30001
server total descriptors : 30009
generator TCP descriptors : 30000
system-wide kern.openfiles : 60115 / kern.maxfiles=400000
kern.maxfilesperproc : 200000
shell RLIMIT_NOFILE (ulimit -n) : 200000
-- socket buffers / mbufs --
577/1973/2550 mbufs in use (current/cache/total)
0/762/762/262144 mbuf clusters in use (current/cache/total/max)
320/696/1016/126883 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37595 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21147 16k jumbo clusters in use (current/cache/total/max)
1424K/4801K/6225K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sfbufs denied
0 requests for sfbufs delayed
kern.ipc.maxsockbuf : 2097152
kern.ipc.maxsockets : 300000
net.inet.tcp.sendspace/recvspace: 32768/65536
-- system memory --
0 0 0 638M 3.4G 8 0 0 0 0 10 0 0 49 96 107 0 0 100
4263751680 901899

CSV row appended to /tmp/bonus.csv:
0/0/0/21147,60007
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #
```

```
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # src/tools/measure.sh 40000 | tee /tmp/measure_40000.txt
=====
BONUS MEASUREMENT label=40000 2026-09-10 18:07:25
server pid=1791 generator pid=1793
=====
-- connections --
established to server port 5000 : 40001
net.inet.tcp.pcbcount (all PCBs): 80007 # ~2x EST on loopback
-- server memory / cpu --
pid=rss=,vsz=,%cpu=,%mem=,time=,nlwp= rss= KB vsz= KB cpu=% mem=% time= threads=
pid=1791 rss= KB vsz= KB cpu=% mem=% time= threads=
-- kernel UMA zones (ITEM / USED / SIZE) --
ITEM SIZE LIMIT USED FREE REQ FAIL SLEEP XDOM
mbuf_packet: 256, 0, 0, 762, 8007, 0, 0, 0
mbuf: 256,1624107, 577, 1211, 1752087, 0, 0, 0
mbuf_cluster: 2048, 262144, 762, 0, 762, 0, 0, 0
mbuf_jumbo_page: 4096, 126883, 320, 696, 94604, 0, 0, 0
mbuf_jumbo_9k: 9216, 37595, 0, 0, 0, 0, 0, 0
mbuf_jumbo_16k: 16384, 21147, 0, 0, 0, 0, 0, 0
socket: 1152, 300000, 80030, 239, 454873, 0, 0, 0
tcp_inpcb: 1320, 0, 80007, 51, 452471, 0, 0, 0
tcp_inpcb ports: 32, 0, 40001, 445, 226267, 0, 0, 0
tcp_log: 408,500000, 0, 0, 0, 0, 0, 0
tcp_log_id_bucket: 176, 0, 0, 0, 0, 0, 0, 0
tcp_log_id_node: 120, 0, 0, 0, 0, 0, 0, 0
(kernel bytes for a zone = USED x SIZE; subtract your idle baseline)
-- file descriptors --
server TCP descriptors : 40001
server total descriptors : 40009
generator TCP descriptors : 40000
system-wide kern.openfiles : 80115 / kern.maxfiles=400000
kern.maxfilesperproc : 200000
shell RLIMIT_NOFILE (ulimit -n) : 200000
-- socket buffers / mbufs --
577/1973/2550 mbufs in use (current/cache/total)
0/762/762/262144 mbuf clusters in use (current/cache/total/max)
320/696/1016/126883 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37595 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21147 16k jumbo clusters in use (current/cache/total/max)
1424K/4801K/6225K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sfbufs denied
0 requests for sfbufs delayed
kern.ipc.maxsockbuf : 2097152
kern.ipc.maxsockets : 300000
net.inet.tcp.sendspace/recvspace: 32768/65536
-- system memory --
0 0 0 638M 3.4G 8 0 0 0 0 10 0 0 183 408 352 0 0 100
4263751680 885903

CSV row appended to /tmp/bonus.csv:
0/0/0/21147,80007
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #
```



```
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # src/tools/measure.sh 50000 | tee /tmp/measure_50000.txt
=====
BONUS MEASUREMENT label=50000 2026-09-10 18:10:50
server pid=1862 generator pid=1864
=====
-- connections --
established to server port 5000 : 50001
net.inet.tcp.pcbcount (all PCBs): 112338 # ~2x EST on loopback
-- server memory / cpu --
pid=rss=,vsz=,%cpu=,%mem=,time=,nlwp= rss= KB vsz= KB cpu=% mem=% time= threads=
pid=1862 rss= KB vsz= KB cpu=% mem=% time= threads=
-- kernel UMA zones (ITEM / USED / SIZE) --
ITEM SIZE LIMIT USED FREE REQ FAIL SLEEP XDOM
mbuf_packet: 256, 0, 0, 762, 8083, 0, 0, 0
mbuf: 256,1624107, 577, 1211, 2224877, 0, 0, 0
mbuf_cluster: 2048, 262144, 762, 0, 762, 0, 0, 0
mbuf_jumbo_page: 4096, 126883, 320, 696, 98061, 0, 0, 0
mbuf_jumbo_9k: 9216, 37595, 0, 0, 0, 0, 0, 0
mbuf_jumbo_16k: 16384, 21147, 0, 0, 0, 0, 0, 0
socket: 1152, 300000, 112361, 416, 554883, 0, 0, 0
tcp_inpcb: 1320, 0, 112338, 42, 552474, 0, 0, 0
tcp_inpcb ports: 32, 0, 54668, 772, 268603, 0, 0, 0
tcp_log: 408,500000, 0, 0, 0, 0, 0, 0
tcp_log_id_bucket: 176, 0, 0, 0, 0, 0, 0, 0
tcp_log_id_node: 120, 0, 0, 0, 0, 0, 0, 0
(kernel bytes for a zone = USED x SIZE; subtract your idle baseline)
-- file descriptors --
server TCP descriptors : 50001
server total descriptors : 50009
generator TCP descriptors : 50000
system-wide kern.openfiles : 100115 / kern.maxfiles=400000
kern.maxfilesperproc : 200000
shell RLIMIT_NOFILE (ulimit -n) : 200000
-- socket buffers / mbufs --
579/1971/2550 mbufs in use (current/cache/total)
0/762/762/262144 mbuf clusters in use (current/cache/total/max)
320/696/1016/126883 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37595 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21147 16k jumbo clusters in use (current/cache/total/max)
1424K/4800K/6225K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sfbufs denied
0 requests for sfbufs delayed
kern.ipc.maxsockbuf : 2097152
kern.ipc.maxsockets : 300000
net.inet.tcp.sendspace/recvspace: 32768/65536
-- system memory --
0 0 0 642M 3.3G 8 0 0 0 0 10 0 0 179 296 346 0 0 100
4263751680 862031

CSV row appended to /tmp/bonus.csv:
0/0/0/21147,112338
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #
```

```
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # src/tools/measure.sh 60000 | tee /tmp/measure_60000.txt
=====
BONUS MEASUREMENT label=60000 2026-09-10 18:11:52
server pid=1933 generator pid=1935
=====
-- connections --
established to server port 5000 : 60001
net.inet.tcp.pcbcount (all PCBs): 120007 # ~2x EST on loopback
-- server memory / cpu --
pid=rss=,vsz=,%cpu=,%mem=,time=,nlwp= rss= KB vsz= KB cpu=% mem=% time= threads=
pid=1933 rss= KB vsz= KB cpu=% mem=% time= threads=
-- kernel UMA zones (ITEM / USED / SIZE) --
ITEM SIZE LIMIT USED FREE REQ FAIL SLEEP XDOM
mbuf_packet: 256, 0, 0, 762, 8115, 0, 0, 0
mbuf: 256,1624107, 577, 1211, 2648869, 0, 0, 0
mbuf_cluster: 2048, 262144, 762, 0, 762, 0, 0, 0
mbuf_jumbo_page: 4096, 126883, 320, 696, 101197, 0, 0, 0
mbuf_jumbo_9k: 9216, 37595, 0, 0, 0, 0, 0, 0
mbuf_jumbo_16k: 16384, 21147, 0, 0, 0, 0, 0, 0
socket: 1152, 300000, 120030, 118, 674891, 0, 0, 0
tcp_inpcb: 1320, 0, 120007, 95, 672475, 0, 0, 0
tcp_inpcb ports: 32, 0, 60001, 731, 328603, 0, 0, 0
tcp_log: 408,500000, 0, 0, 0, 0, 0, 0
tcp_log_id_bucket: 176, 0, 0, 0, 0, 0, 0, 0
tcp_log_id_node: 120, 0, 0, 0, 0, 0, 0, 0
(kernel bytes for a zone = USED x SIZE; subtract your idle baseline)
-- file descriptors --
server TCP descriptors : 60001
server total descriptors : 60009
generator TCP descriptors : 60000
system-wide kern.openfiles : 120115 / kern.maxfiles=400000
kern.maxfilesperproc : 200000
shell RLIMIT_NOFILE (ulimit -n) : 200000
-- socket buffers / mbufs --
577/1973/2550 mbufs in use (current/cache/total)
0/762/762/262144 mbuf clusters in use (current/cache/total/max)
320/696/1016/126883 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37595 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21147 16k jumbo clusters in use (current/cache/total/max)
1424K/4801K/6225K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sfbufs denied
0 requests for sfbufs delayed
kern.ipc.maxsockbuf : 2097152
kern.ipc.maxsockets : 300000
net.inet.tcp.sendspace/recvspace: 32768/65536
-- system memory --
0 0 0 648M 3.3G 8 0 0 0 0 11 0 0 149 318 294 0 0 99
4263751680 855127

CSV row appended to /tmp/bonus.csv:
0/0/0/21147,120007
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #
```



```

root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 # src/tools/measure.sh 70000 | tee /tmp/measure_70000.txt
=====
BONUS MEASUREMENT label=70000 2026-09-10 18:12:53
server pid=1996 generator pid=1998
=====
-- connections --
established to server port 5000 : 70001
net.inet.tcp.pcbcount (all PCBs): 140007 # ~2x EST on loopback
-- server memory / cpu --
pid=rss=,vsz=,%cpu=,%mem=,time=,nlwp= rss= KB vsz= KB cpu=% mem=% time= threads=
pid=1996 rss= KB vsz= KB cpu=% mem=% time= threads=
-- kernel UMA zones (ITEM / USED / SIZE) --
ITEM SIZE LIMIT USED FREE REQ FAIL SLEEP XDOM
mbuf_packet: 256, 0, 0, 762, 8143, 0, 0, 0
mbuf: 256,1624107, 579, 1209, 3106199, 0, 0, 0
mbuf_cluster: 2048, 262144, 762, 0, 762, 0, 0, 0
mbuf_jumbo_page: 4096, 126883, 320, 696, 103952, 0, 0, 0
mbuf_jumbo_9k: 9216, 37595, 0, 0, 0, 0, 0, 0
mbuf_jumbo_16k: 16384, 21147, 0, 0, 0, 0, 0, 0
socket: 1152, 300000, 140030, 432, 814898, 0, 0, 0
tcp_inpcb: 1320, 0, 140007, 75, 812476, 0, 0, 0
tcp_inpcb ports: 32, 0, 64513, 629, 393115, 0, 0, 0
tcp_log: 408,500000, 0, 0, 0, 0, 0, 0
tcp_log_id_bucket: 176, 0, 0, 0, 0, 0, 0, 0
tcp_log_id_node: 120, 0, 0, 0, 0, 0, 0, 0
(kernel bytes for a zone = USED x SIZE; subtract your idle baseline)
-- file descriptors --
server TCP descriptors : 70001
server total descriptors : 70009
generator TCP descriptors : 70000
system-wide kern.openfiles : 140115 / kern.maxfiles=400000
kern.maxfilesperproc : 200000
shell RLIMIT_NOFILE (ulimit -n) : 200000
-- socket buffers / mbufs --
577/1973/2550 mbufs in use (current/cache/total)
0/762/762/262144 mbuf clusters in use (current/cache/total/max)
320/696/1016/126883 4k (page size) jumbo clusters in use (current/cache/total/max)
0/0/0/37595 9k jumbo clusters in use (current/cache/total/max)
0/0/0/21147 16k jumbo clusters in use (current/cache/total/max)
1424K/4801K/6225K bytes allocated to network (current/cache/total)
0/0/0 requests for mbufs denied (mbufs/clusters/mbuf+clusters)
0/0/0 requests for mbufs delayed (mbufs/clusters/mbuf+clusters)
0/0/0 requests for jumbo clusters delayed (4k/9k/16k)
0/0/0 requests for jumbo clusters denied (4k/9k/16k)
0 requests for sbufs denied
0 requests for sbufs delayed
kern.ipc.maxsockbuf : 2097152
kern.ipc.maxsockets : 300000
net.inet.tcp.sendspace/recvspace: 32768/65536
-- system memory --
0 0 0 646M 3.2G 8 0 0 0 0 11 0 0 31 56 78 0 0 100
4263751680 837863

CSV row appended to /tmp/bonus.csv:
0/0/0/21147,140007
root@exchange:/home/cooluser1/A2_2024CS10573_2024CS10836 #

```

9.5 Answers

1. Is the server able to maintain all requested connections at each count? If not, at which count does it first fail?

Yes, at every level from 10,000 through 70,000. The **FAIL** column of **vmstat -z** remained 0 for every zone throughout, and the generator reported no failed **socket()** or **connect()** call. No failure point was reached within the target range **after** the configuration of §9.2 was applied.

9.5.1 The deliberate failure run

Before that configuration, the first failure was reproducible and instructive. Running the generator against a **single** source address terminated with

```
[gen] STOPPED: connect(): EADDRNOTAVAIL (ephemeral ports exhausted) (errno
```

49)

at approximately the size of the ephemeral port pool, well short of 70,000. This was captured alongside

```
sysctl net.inet.ip.portrange.first net.inet.ip.portrange.last
```

so that the observed ceiling can be checked directly against the configured range. The failure is one of connection *identity*, not of any resource: no memory, descriptor or CPU limit was anywhere near its bound at that point. Re-running with `--src 127.0.0.1,127.0.0.2,127.0.0.3` passes straight through that number, which is the controlled demonstration that the 4-tuple space — not resource consumption — is what first limits a single-port server.

2. What is the first significant bottleneck encountered, and what in the table supports that?

The binding constraint was **TCP connection identity — ephemeral port exhaustion — not memory, CPU, or file descriptors**. Each column rules out an alternative:

- *CPU* is eliminated: it never exceeded 0.6 % and was 0.2 % at 70,000, because idle sockets generate no `kqueue` events and the process is asleep in `kevent()` essentially all of the time.
- *Userspace memory* is eliminated: RSS grew from 5,304 KB to 15,448 KB across a sevenfold increase in connections — about **170 bytes per connection** — a negligible fraction of 2 GB.
- *Socket-buffer memory* is eliminated: mbuf cluster usage stayed at 0, since idle connections allocate no packet buffers.
- *File descriptors* were not the first limit either, once `kern.maxfiles` and `kern.maxfilesperproc` were raised: system-wide usage peaked at 150,111 against a 400,000 ceiling.

What remains, and what actually stops the ladder without intervention, is the size of the 4-tuple space: the `tcp_inpcb` zone tracks connections one-for-one up to 60,002 entries and then flattens at 60,743 and 61,442 for the last two levels, consistent with the ~64,000-port pool for a single source address saturating. Without additional loopback source addresses the test would have failed around 64,000 connections regardless of available memory.

3. How does the concurrency/I/O design contribute to the observed bottleneck?

The Exchange Server is a single-threaded, non-blocking `kqueue` event loop. It creates no thread or process per connection. Per connection it costs one file descriptor, one kernel socket structure and its PCB, two `knote` registrations (`EVFILT_READ` enabled, `EVFILT_WRITE` disabled), and one `Conn` object — a `LineBuffer`, an `OutBuffer`, a role, and two subscription flags, measured at ~170 bytes including the hash-map node.

The design therefore contributes essentially **nothing** to the bottleneck that limits the ladder: what limits it is the kernel's socket, PCB and port-space accounting, which any design must pay regardless of how it multiplexes I/O. The design's own cost scales with the number of *ready* connections per wakeup, not with the number registered.

4. Would changing the I/O/concurrency mechanism help?

No — the comparison runs the other way, and each alternative fails for a different reason.

A **thread-per-connection** design would need 70,000 threads. FreeBSD caps threads per process at `kern.threads.max_threads_per_proc`, so `pthread_create` fails at roughly 1,500 — about 2 % of the target — long before any socket or descriptor limit is approached. Even setting that cap aside, the default per-thread stack reservation of 2 MB implies ~140 GB of virtual address space for 70,000 threads, and the scheduler and context-switch overhead would scale with the number of connections rather than with the number of active ones. A minimal thread-per-connection server (`tools/threaded_server.cpp`) was built solely to measure this and reports the exact failure count.

`select()` is worse still: it is bounded by `FD_SETSIZE` (1024 by default), which is 1.4 % of the target, and it requires the kernel to scan the entire descriptor set on every call, making each wakeup $O(n)$.

`poll()` removes the `FD_SETSIZE` limit but keeps the $O(n)$ scan and additionally copies the entire descriptor array across the syscall boundary on every call. At 70,000 mostly-idle descriptors, that is the difference between examining 70,000 entries per wakeup and being handed only the one that changed.

`kqueue` registers interest once and returns only the descriptors whose state changed, so its cost scales with *active* connections rather than *open* ones. For a workload of mostly-idle sockets this is the best available choice, which is why no change of mechanism would improve the measured ceiling.

5. Quantify the trade-off.

Two configuration changes were made, and their effects differ instructively.

Change A — socket buffer sizes. `net.inet.tcp.sendspace` and `recvspace` were reduced from their defaults to 4 KB with auto-tuning disabled. This bounds the worst-case *reservation* per connection: on loopback each connection has four buffers (send and receive at each of two endpoints), so at a 32 KB default the reservation ceiling for 70,000 connections is ~9 GB, against ~1.1 GB at 4 KB. The measurement qualifies this: because the connections are idle, actual mbuf usage stayed at 0 throughout, so this change protected against a worst case rather than reducing observed consumption. It is a cheap safety margin for an idle-connection workload and is safe precisely because no data flows.

Change B — descriptor and port limits. Raising `kern.maxfiles`, `kern.maxfilesperproc`, `kern.ipc.maxsockets` and widening `net.inet.ip.portrange`, together with additional loopback source addresses, is what actually moved the ceiling — from failure at ~55,000–64,000 with `EADDRNOTAVAIL/EMFILE` to a clean 70,000.

The contrast that matters. Shrinking per-thread stacks in the thread-per-connection design reduces virtual address space per connection but does **not** move `kern.threads.max_threads_per_proc`, so that design's ceiling is unchanged by tuning. The `kqueue` design's ceiling, by contrast, responded directly to configuration. That asymmetry — one design limited by a fixed structural cap, the other by adjustable accounting — is the substantive finding of this bonus.

10. Implementation Decisions

Concurrency and I/O: a single-threaded `kqueue` event loop

The listening socket and every accepted socket are registered with one `kqueue` instance, and the loop blocks in `kevent()` with a NULL timeout until the kernel reports readiness. There is no thread or process

per connection.

This was chosen because the assignment's workload is a large number of connections of which only a few are active at any instant, which is exactly the case readiness notification is designed for. Cost scales with the number of *ready* descriptors rather than the number *open*, so an idle client costs no CPU at all — demonstrated directly in Experiment 4, where `procstat -k` shows the thread asleep in `kqueue_scan` while another client is served in under a millisecond, and again in the bonus, where 70,000 idle connections held CPU at 0.2 %. A single thread also removes any need for locking around the order book, which keeps the matching engine straightforwardly correct.

Each accepted descriptor is registered **twice at accept time** — `EVFILT_READ` enabled and `EVFILT_WRITE` disabled — and write interest is subsequently toggled with `EV_ENABLE/EV_DISABLE` rather than add/delete. This halves the syscalls on the hot path and avoids `ENOENT` from deleting a filter that was never registered.

Non-blocking sockets and per-connection output buffers

Every accepted socket is set `O_NONBLOCK` and the server never blocks in `send()`. All server→client bytes pass through a single `queue_out() → try_flush()` path; there is no bare `send()` anywhere else. On **EAGAIN** the remainder stays in that connection's own `OutBuffer`, `EVFILT_WRITE` is enabled for that descriptor alone, and control returns to the event loop. Experiment 7 measures this directly: 4519 refusals, 76,823 bytes buffered, and zero impact on the other three clients.

`OutBuffer` consumes bytes by advancing an offset rather than erasing from the front of the string, because front-erasure is $O(\text{remaining})$ and would make writes to a slow reader quadratic. The buffer is capped at 8 MB per connection; a client exceeding it is disconnected, so a subscriber that never reads cannot grow the server heap without bound.

Message framing

Each connection owns a `LineBuffer`. Whatever `recv()` returns is appended, and the server extracts complete lines in a loop that scans for `\n`. Because the buffer persists across calls, an incomplete message simply waits for the rest to arrive (Experiment 3); because extraction loops, several messages arriving in one `recv()` are all handled. A trailing `\r` is stripped so that `nc` and `telnet`, which send CRLF, work unmodified.

Connection identity separate from the file descriptor

Orders store the submitting trader's logical `owner_id`, a monotonically increasing counter that is never reused — never a file descriptor, since the kernel recycles descriptor numbers aggressively. This is what makes handout §2.6 work correctly: resting orders are deliberately left in the book when their owner disconnects, and if such an order later executes, subscribed Market-Data Clients still receive **TRADE** while the **BOUGHT/SOLD** for the departed trader is silently skipped. Had orders been keyed by descriptor, a newly accepted client inheriting a recycled fd would have received another trader's execution notifications.

Client roles

The protocol has no handshake announcing the client type, so the role is inferred from the first command that succeeds: `LOGIN` promotes a connection from `Unknown` to `Trader`, `SUBSCRIBE/UNSUBSCRIBE` to `MarketData`. Thereafter each handler rejects a command not permitted for the role with `ERROR not permitted for this client type` and **does not** disconnect, since the handout requires only that the command be refused.

Validation

Numeric fields are parsed by a hand-written validator rather than `stoi/atoi`: length is checked before conversion, every character must be a decimal digit, and only then is `strtoull` applied and range-checked. This rejects `-5`, `+7`, `2.5`, `1e3`, `0x10`, `12abc` and leading/trailing whitespace uniformly. Validation completes before the exchange is touched, so a rejected order never consumes an order ID — the handout requires that such an order "must not be accepted or executed", and a skipped ID would be externally detectable.

Other decisions

- `SO_REUSEADDR` on the listening socket, so the server can be restarted while old connections linger in `TIME_WAIT`. It does not permit two live listeners on the same endpoint.
- `SIGPIPE` is ignored, so writing to a connection whose peer has departed returns `EPIPE` rather than killing the process.
- `shutdown(fd, SHUT_WR)` is issued before `close()` on every teardown, which gives a clean `FIN` for Experiment 2 and satisfies the handout's requirement to use `shutdown` where appropriate.
- **Deferred connection reaping.** `close_conn()` marks a connection dead and pushes its descriptor onto a `dead_` list; the actual `close()` and map erasure happen once per event-loop iteration after all events are processed. This prevents iterator invalidation when a broadcast in `publish()` closes a connection while iterating the connection map, and it also prevents a freed descriptor number from being recycled by an `accept()` in the same event batch while stale events for the old descriptor are still pending.
- Clients use blocking sockets with `select()` over `stdin` and the socket, which is simpler and fully permitted; the multiplexing is necessary so that an asynchronous `BOUGHT/SOLD` can be printed while the user sits at the prompt.
- Clients call `setvbuf(stdout, nullptr, _IONBF, 0)`, because `experiment.py` attaches client `stdout` to a pipe, on which C++ would otherwise switch to 4 KB block buffering and the harness would observe nothing.
- Leading zeros are accepted in numeric fields (`0100 == 100`) as unambiguous decimal, and `CANCEL` is restricted to the trader that submitted the order. Both are documented in the README so they are not read as oversights.

11. Appendix A — Tools and What Each One Observes

Tool	Layer observed	Used in
<code>tcpdump -i lo0</code>	the wire: flags, sequence numbers, window sizes, timestamps	2, 3, 4, 6, 7, 8

Tool	Layer observed	Used in
<code>netstat -an -p tcp</code>	PCB table: 4-tuples, states, <code>Recv-Q/Send-Q</code>	all
<code>netstat -an -p tcp -x</code>	extended socket state: <code>R-HIWA/S-HIWA</code> , <code>R-BCNT</code> , <code>persist</code> , <code>rcvtime</code>	5, 7, 8
<code>netstat -aLn -p tcp</code>	listen queue depth (<code>qlen/incqlen/maxqlen</code>)	1
<code>netstat -m</code>	mbuf and cluster allocation	bonus
<code>sockstat -4</code>	process ↔ socket binding, listening vs connected	1, 5, 8
<code>procstat -f</code>	per-process file-descriptor table, including the kqueue fd	1, 4, 5, 8, bonus
<code>procstat -k</code>	kernel stack: exactly where the thread is sleeping	4
<code>procstat -b</code>	binary path, confirming <code>exec</code> in the launcher	1
<code>ktrace / kdump</code>	syscall sequence and return values per descriptor	3, 4, 5, 6, 8
<code>vmstat -z</code>	UMA zone usage: per-connection kernel structure cost	bonus
<code>top, ps</code>	RSS, VSZ, CPU, thread count	7, bonus
<code>sysctl</code>	kernel tunables and counters	2, 7, bonus

Note on `procstat -k`. It must be given a valid PID; capture the PID into a variable at the start of the run rather than re-running `pgrep` inline, since an inline `pgrep` that matches nothing turns the command into a bare `procstat -k` and prints a usage message instead of a stack.

12. Appendix B — Reproduction Instructions

```
# build
make

# a typical experiment, in five terminals (T2-T4 first, T1 last)
~/tools/poll-tcp.sh 0.2 /tmp/expN.states # T2
tcpdump -l -i lo0 -n -S -vv -tttt 'tcp port 5000' > /tmp/expN.txt # T3
while ! PID=$(pgrep -n exchange_server); do sleep 0.05; done \
    && ktrace -i -f /tmp/expN.kt -p $PID -t c # T4
SX_VERBOSE=1 python3 -u experiment.py N 2>&1 | tee /tmp/expN.log # T1
# ... T5 for anything that must be sampled mid-phase ...
ktrace -C && kdump -f /tmp/expN.kt -T | egrep 'kevent|recvfrom' # T4,
after

# experiment 7 additionally requires clamped socket buffers, or flow
control
# never engages at this data volume (see the control run in §7)
sysctl kern.ipc.maxsockbuf=16384
```



```

sysctl net.inet.tcp.recvspace=4096 net.inet.tcp.sendspace=4096
sysctl net.inet.tcp.recvbuf_auto=0 net.inet.tcp.sendbuf_auto=0
# ... restore the §0.4 defaults afterwards, before running experiment 8
...

# bonus – repeat this 4-step cycle for each rung: 10000 ... 70000
make bonus
ifconfig lo0 alias 127.0.0.2/32 && ifconfig lo0 alias 127.0.0.3/32

# T1: restart the server clean before every rung
pkill exchange_server; ulimit -n 200000
./server/run-server 127.0.0.1 5000 > /dev/null 2>&1

# T2: hold N idle connections
ulimit -n 200000
./connflood 127.0.0.1 5000 <N> --src 127.0.0.1,127.0.0.2,127.0.0.3 \
  --report 2500 --batch 64 2>&1 | tee /tmp/gen_<N>.log

# T3: measure while they are held
src/tools/measure.sh <N> | tee /tmp/measure_<N>.txt

# T2: Ctrl-C, then confirm the drain
netstat -an -p tcp | awk '$4 ~ /\..5000$/ && $6=="ESTABLISHED"' | wc -l

```

Between every run:

```

pkill exchange_server
ktrace -C
pgrep -f exchange_server          # must print nothing before the next
experiment

```

A stale server from a previous run holds port 5000, and the next harness invocation will then either fail to bind or connect to the wrong process.